

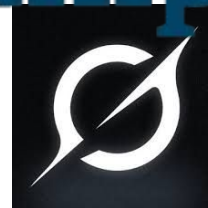
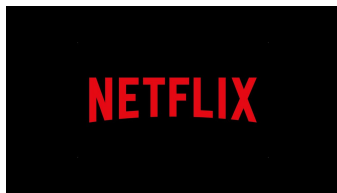
Unidad 1

Conceptos básicos del ML. Aprendizaje supervisado, no supervisado, semi supervisado. Aprendizaje profundo. Entrenamiento, validación y evaluación. Sobreajuste/subajuste + regularización (nivel conceptual). Principios del aprendizaje. Data leakage. Cross-validation espacial. Validación temporal.

Introducción

Generalidades

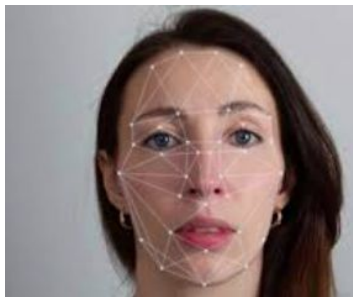
Machine learning en la vida cotidiana



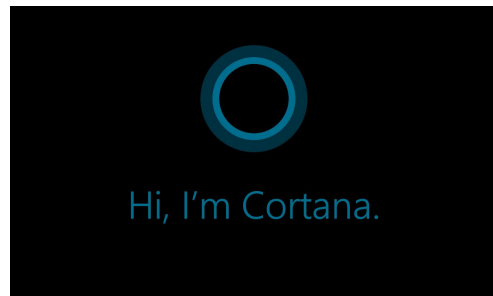
Modelos de clasificación



Asistente de
VOZ



Sistemas de
recomendación



Inteligencia artificial en la vida cotidiana

- chatGPT 5
- Sora AI
- Nano Banana
- Veo3

Large Language
Models (LLMs)

Modelos
generativos

Agentes



ChatGPT



<https://cdn.openai.com/sora/videos/mitten-astronaut.mp4>

<https://lmarena.ai/>



<https://storage.googleapis.com/gweb-gemini-cdn/gemini/uploads/b06ee5fbeac029f214aa25285980ce718bcc0f4e.compressed.mp4>



Fuente:

<https://ai.google.dev/gemini-api/docs/image-generation?hl=es-419>

Definición de aprendizaje automático



El aprendizaje automático se trata de **extraer conocimiento de los datos**.

Andreas C Mueller - Introduction to Machine Learning with Python

Definimos el aprendizaje automático como un **conjunto de métodos que pueden automáticamente detectar patrones en los datos, y luego utilizar los patrones descubiertos para predecir datos en el futuro, o realizar otros tipos de decisión bajo cierta incertidumbre**.

Kevin P Murphy - Machine Learning A Probabilistic Perspective

El aprendizaje automático se trata de la **construcción de algoritmos, que, para ser útiles, se basan en la colección de ejemplos de algún fenómeno**. Los ejemplos pueden provenir de la naturaleza, ser recolectados por humanos o generados por otro algoritmo.

El aprendizaje automático también se puede definir como el proceso de resolución de un **problema práctico**, mediante, 1) recopilar un conjunto de datos y 2) entrenar algorítmicamente un modelo estadístico basado en ese conjunto de datos.

Andriy Burkov - Machine Learning Engineering

Definición de aprendizaje automático



Se dice que un programa de computadora aprende de una **experiencia E**, con respecto a alguna **clase de tareas T** y una **medida de desempeño P**, si su desempeño en las tareas de T, medido por P, mejora con la experiencia E.

En términos matemáticos, el ML busca encontrar una función de aproximación f tal que $y = f(x)$, donde x es el vector de entrada y y es el resultado, minimizando una función de pérdida definida.

Por ejemplo, un programa de computadora que aprende a jugar a las damas podría mejorar su **desempeño** (medido por su capacidad para ganar en la clase de **tareas** que implican jugar partidas de damas) a través de la **experiencia** obtenida al jugar partidas contra sí mismo.

En general, para tener un problema de aprendizaje bien definido, debemos identificar estas tres características:

1. La clase de **tareas** (*Tasks*).
2. La medida de **desempeño** (*Performance*) que debe mejorarse.
3. La fuente de **experiencia** (*Experience*).

- Tom Mitchell. *Machine learning*

Definición de aprendizaje profundo



El **aprendizaje profundo** (*deep learning*) es un subconjunto del **aprendizaje automático** (*machine learning*), que es un campo dedicado al estudio y desarrollo de máquinas capaces de aprender (a veces con el objetivo de alcanzar eventualmente una **inteligencia artificial general**).

En la industria, el aprendizaje profundo se utiliza para resolver tareas prácticas en una variedad de campos, tales como:

- **Visión por computadora** (imágenes).
- **Procesamiento de lenguaje natural** (texto).
- **Reconocimiento automático del habla** (audio).

En resumen, el aprendizaje profundo es un subconjunto de métodos dentro de la caja de herramientas del aprendizaje automático que utiliza principalmente **redes neuronales artificiales**, las cuales son una clase de algoritmos inspirados libremente en el cerebro humano.

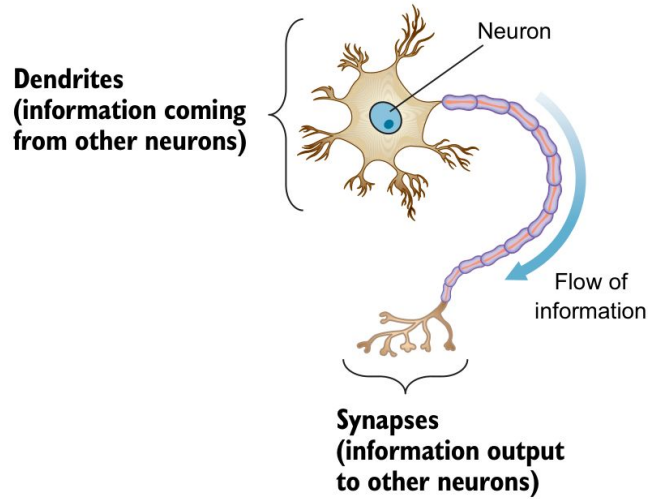
- Andrew Trask. *Grokking Machine Learning*

El **aprendizaje profundo** (*deep learning*) es un subcampo específico del aprendizaje automático: un nuevo enfoque para aprender representaciones de datos que pone énfasis en el aprendizaje de capas sucesivas de representaciones cada vez más significativas. El término "profundo" en "aprendizaje profundo" no hace referencia a ningún tipo de comprensión profunda alcanzada por este enfoque; más bien, representa la idea de capas sucesivas de representaciones. Cuántas capas contribuyen a un modelo de los datos es lo que se denomina la **profundidad** del modelo. Otros nombres apropiados para este campo podrían haber sido **aprendizaje de representaciones por capas** o **aprendizaje de representaciones jerárquicas**.

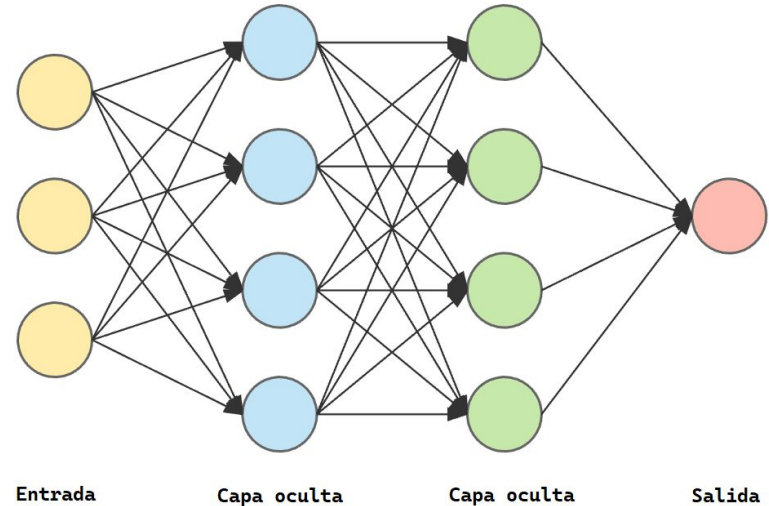
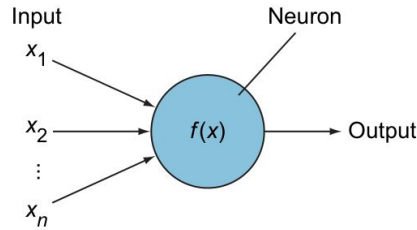
- Francois Chollet - *Deep learning with Python*

Aprendizaje profundo y redes neuronales

Biological neuron



Artificial neuron



Aprendizaje automático vs aprendizaje profundo

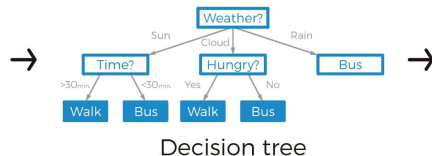
La distinción técnica más rigurosa reside en la **extracción de características (Feature Extraction)**:

- En **ML clásico**, la extracción de características es **exógena** (externa al modelo, realizada por un experto).
- En **DL**, la extracción de características es **endógena** (el modelo aprende la representación óptima de los datos brutos por sí mismo mediante procesos como el *backpropagation*).

Machine Learning



Input

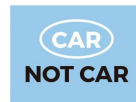
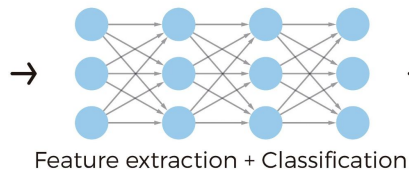


Output

Deep Learning



Input



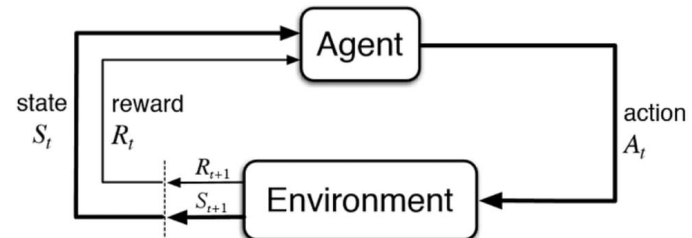
Output

Definición de aprendizaje por refuerzo

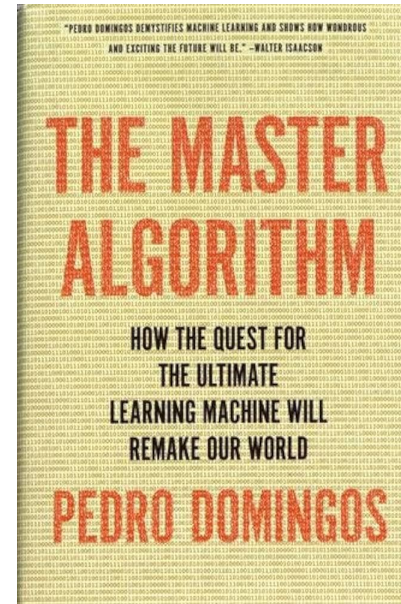
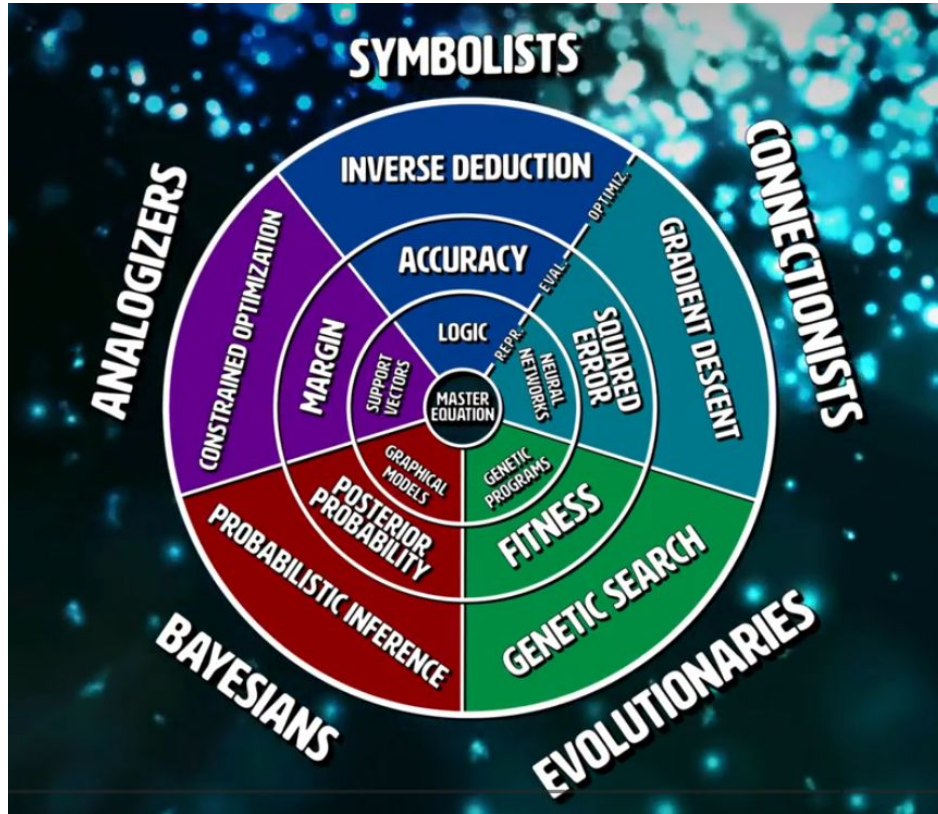
El **aprendizaje por refuerzo** (*reinforcement learning*) es un subcampo del aprendizaje automático donde la máquina (llamada **agente**) "vive" en un entorno y es capaz de percibir el estado de dicho entorno como un vector de características. La máquina puede ejecutar acciones en estados no terminales. Diferentes acciones aportan diferentes recompensas y también podrían trasladar a la máquina a otro estado del entorno. Un objetivo común de un algoritmo de aprendizaje por refuerzo es aprender una **política óptima**.

Una **política óptima** es una función (similar al modelo en el aprendizaje supervisado) que toma el vector de características de un estado como entrada y devuelve como resultado una acción óptima para ejecutar en ese estado. La acción es óptima si maximiza la recompensa promedio esperada a largo plazo.

El aprendizaje por refuerzo **resuelve un problema particular donde la toma de decisiones es secuencial y el objetivo es a largo plazo**, como en los juegos, la robótica, la gestión de recursos o la logística.

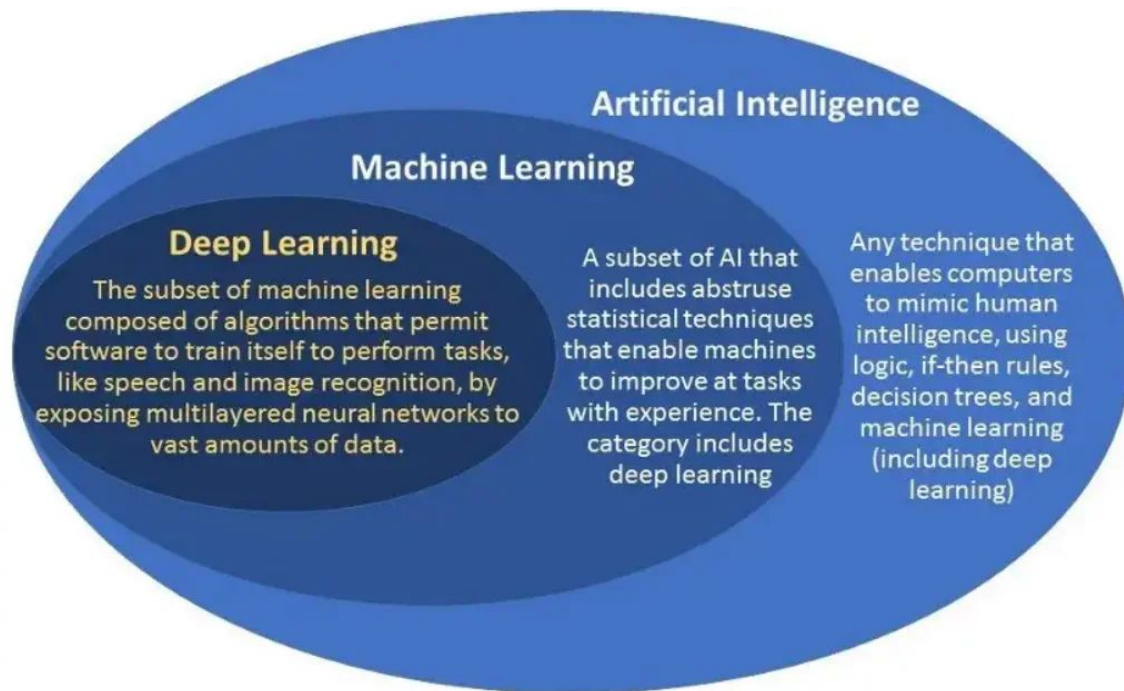


Escuelas dentro del aprendizaje



Resumiendo

- Subcampo de la inteligencia artificial.
- Machine Learning vs Deep Learning
- Ciencia de datos



Aprendizaje automático vs aprendizaje profundo

Característica



Estructura



Intervención humana



Cantidad de datos

Machine Learning

- Utiliza algoritmos estadísticos y matemáticos (Regresión, Árboles de decisión, SVM).



- Requiere que un experto extraiga las características manualmente (Feature Engineering).



- Funciona bien con conjuntos de datos pequeños o medianos.



Deep Learning

- Basado principalmente en Redes Neuronales Artificiales de muchas capas.



- Capaz de aprender y extraer características automáticamente de los datos.



- Requiere grandes volúmenes de datos (Big Data) para ser efectivo.



Aprendizaje automático vs aprendizaje profundo

Característica



Poder de
Cómputo



Tiempo de
Entrenamiento



Interpretabili-
dad

Machine Learning

- Se puede ejecutar en CPUs estándar; es menos exigente.



- Relativamente corto (segundos a horas).



- Generalmente es más fácil entender "por qué" tomó una decisión (Caja blanca).



Deep Learning

- Requiere alta capacidad de cómputo, usualmente GPUs de alto rendimiento.



- Muy largo (días o semanas para modelos complejos).

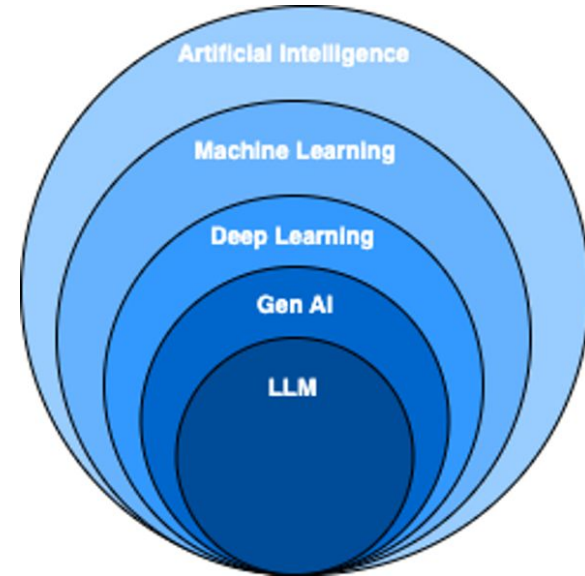
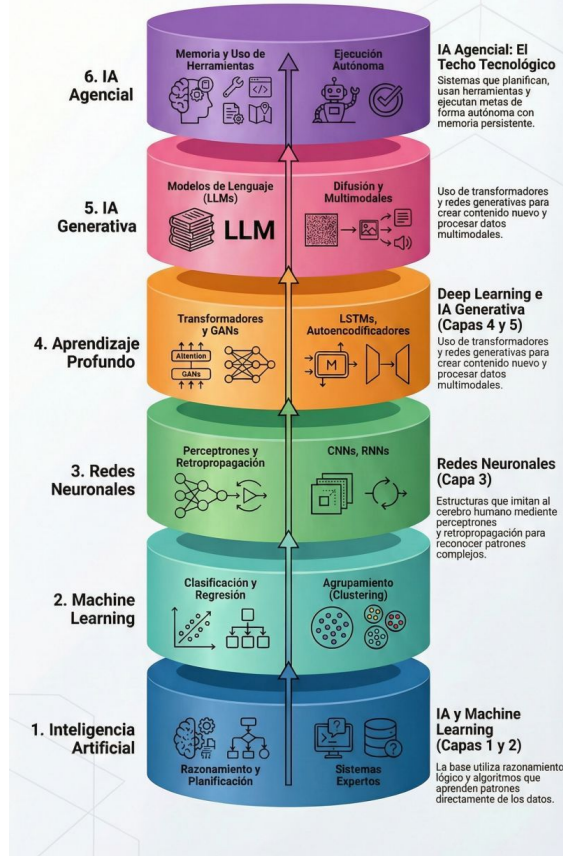


- Es más complejo y difícil de explicar; a menudo se considera una "Caja negra".



Hoy en día

Capas de la Inteligencia Artificial: La Evolución de la Complejidad





Aprendizaje automático y ciencia de datos



Cuando es apropiado ML para abordar un problema

1. Cuando el problema es complejo para ser programado (exceso de reglas para programar).
2. Cuando el problema está cambiando constantemente.
3. Cuando es un problema relacionado a la percepción (visión, lenguaje).
4. Cuando es un fenómeno no estudiado.
5. Cuando el problema tiene un objetivo simple.
6. Cuando es eficiente con respecto al costo.*

Cuando el problema es complejo para ser programado

1. Cuando el problema es complejo para ser programado (exceso de reglas para programar)

Un ejemplo es la **detección de spam**: es imposible escribir el código que implemente una lógica capaz de detectar eficazmente los mensajes de spam y permitir, al mismo tiempo, que los mensajes auténticos lleguen a la bandeja de entrada. Hay demasiados factores a considerar. Por ejemplo, si programas tu filtro de spam para rechazar todos los mensajes de personas que no están en tus contactos, corres el riesgo de perder mensajes de alguien que recibió tu tarjeta de presentación en una conferencia. Si haces una excepción para mensajes que contengan palabras clave específicas relacionadas con tu trabajo, probablemente se te escape un mensaje del profesor de tu hijo, y así sucesivamente.

Si aun así decides programar directamente una solución para ese problema complejo, con el tiempo tendrás en tu código tantas condiciones y excepciones que mantenerlo acabará siendo inviable. En esta situación, **entrenar un clasificador** con ejemplos de "spam" y "no spam" parece lógico y es la única opción viable.

Cuando el problema está cambiando constantemente



2. Cuando el problema está cambiando constantemente.

Algunos problemas pueden cambiar continuamente con el tiempo, de modo que el código de programación debe actualizarse regularmente. Esto genera frustración en los ingenieros de software que trabajan en el problema, aumenta la probabilidad de introducir errores, crea dificultades para combinar la lógica "anterior" con la "nueva" y supone una carga significativa de trabajo en las pruebas y el despliegue de las soluciones actualizadas.

Por ejemplo, podrías tener la tarea de extraer datos específicos (*scraping*) de una colección de páginas web. Supongamos que, para cada página de esa colección, escribes un conjunto de reglas fijas de extracción de la siguiente forma: "toma el tercer elemento `<p>` del `<body>` y luego extrae los datos del segundo `<div>` dentro de ese `<p>`". Si el propietario del sitio web cambia el diseño de una página, los datos que extraes podrían terminar en el segundo o en el cuarto elemento `<p>`, haciendo que tu regla de extracción sea errónea.

La solución de ML: En lugar de reglas rígidas basadas en la posición del elemento (como el "tercer párrafo"), un modelo de Machine Learning podría entrenarse para identificar visualmente o por contexto qué es un "precio" o un "título", sin importar si se movió de lugar en el código HTML.

Cuando el problema está relacionado a la percepción

3. Cuando es un problema relacionado a la percepción (visión, lenguaje).

Hoy en día, es difícil imaginar a alguien intentando resolver **problemas de la percepción** —como el reconocimiento de voz, de imágenes y de video— sin utilizar el aprendizaje automático. Consideremos una imagen: está representada por millones de píxeles. Cada píxel viene definido por tres números: la intensidad de los canales rojo, verde y azul.

En el pasado, los ingenieros intentaban resolver el problema del reconocimiento de imágenes (detectar qué hay en la foto) aplicando "filtros" artesanales a parches cuadrados de píxeles. Si un filtro —por ejemplo, uno diseñado para "detectar" pasto— genera un valor alto al aplicarse a muchos parches de píxeles, mientras que otro filtro, diseñado para detectar pelaje marrón, también devuelve valores altos en muchos parches, entonces podemos decir que hay grandes probabilidades de que la imagen representa a una vaca en un campo (simplificando).

Cuando es un fenómeno no estudiado

4. Cuando es un fenómeno no estudiado.

Si necesitamos realizar predicciones sobre algún fenómeno que **no ha sido bien estudiado científicamente, pero del cual existen ejemplos observables**, entonces el aprendizaje automático podría ser una opción adecuada (y, en algunos casos, la única disponible).

Otro ejemplo de fenómenos observables pero no estudiados son los registros (*logs*) de un sistema informático o una red compleja. Dichos registros son generados por múltiples procesos independientes o interdependientes. Para un ser humano, es difícil hacer predicciones sobre el estado futuro del sistema basándose únicamente en los registros sin tener un modelo de cada proceso y su interdependencia. Si el número de ejemplos de registros históricos es lo suficientemente alto (lo cual suele ser el caso), la máquina puede aprender patrones ocultos en los registros y ser capaz de hacer predicciones sin saber nada sobre cada proceso individual.

Cuando el problema tiene un objetivo simple

5. Cuando el problema tiene un objetivo simple.

El aprendizaje automático es especialmente adecuado para resolver problemas que se pueden formular con un **objetivo simple**: como decisiones de "sí o no" o el cálculo de un solo número. En contraste, no se puede usar el aprendizaje automático para construir un modelo que funcione como un videojuego general, como Mario, o un software de procesamiento de textos, como Word.

Esto se debe a que hay demasiadas decisiones distintas que tomar: qué mostrar, dónde y cuándo, qué debe suceder como reacción a la entrada del usuario, qué escribir o leer en el disco duro, y así sucesivamente; obtener ejemplos que ilustran todas (o incluso la mayoría) de esas decisiones es prácticamente inviable.

Cuando es eficiente con respecto al costo

5. Cuando es eficiente con respecto al costo.

Las tres fuentes principales de costos en el aprendizaje automático son:

- La **recopilación, preparación y limpieza de los datos**.
- El **entrenamiento del modelo**.
- La **construcción y ejecución de la infraestructura** para poner en servicio y monitorear el modelo, así como los recursos humanos para mantenerlo.

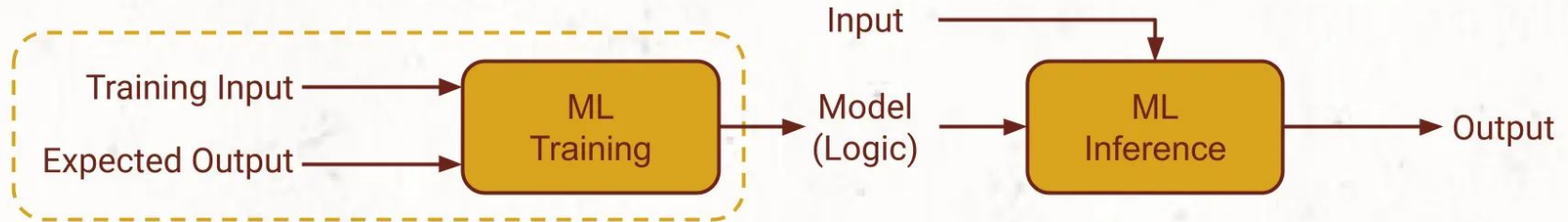
El costo de entrenar el modelo incluye la mano de obra humana y, en algunos casos, el hardware costoso necesario para entrenar modelos profundos (*deep models*). El mantenimiento del modelo incluye el monitoreo continuo y la recolección de datos adicionales para mantener el modelo actualizado.

Aprendizaje automático vs. programación tradicional

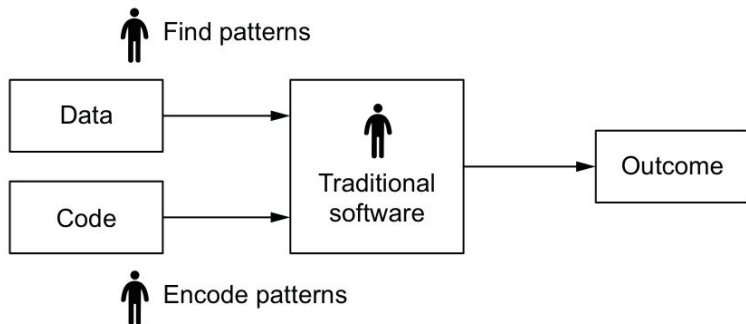
Traditional Programs: Define algo/logic to compute output



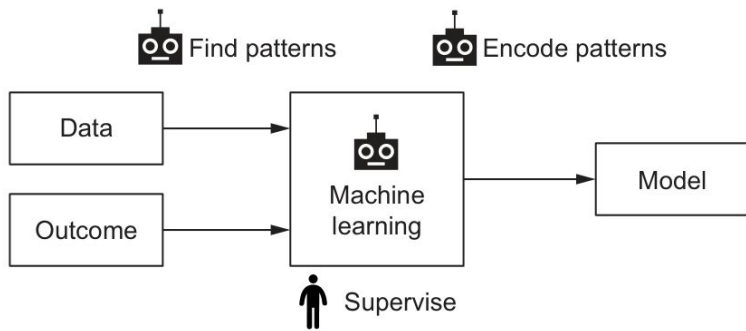
Machine Learning: Learn model/logic from data



Aprendizaje automático vs. programación tradicional



(A) In traditional software we discover patterns manually and encode them using a programming language.



(B) A machine learning system discovers patterns automatically by learning from examples. After training, it produces a model that “knows” these patterns, but we still need to supervise it to make sure the model is correct.

Tipos de aprendizaje

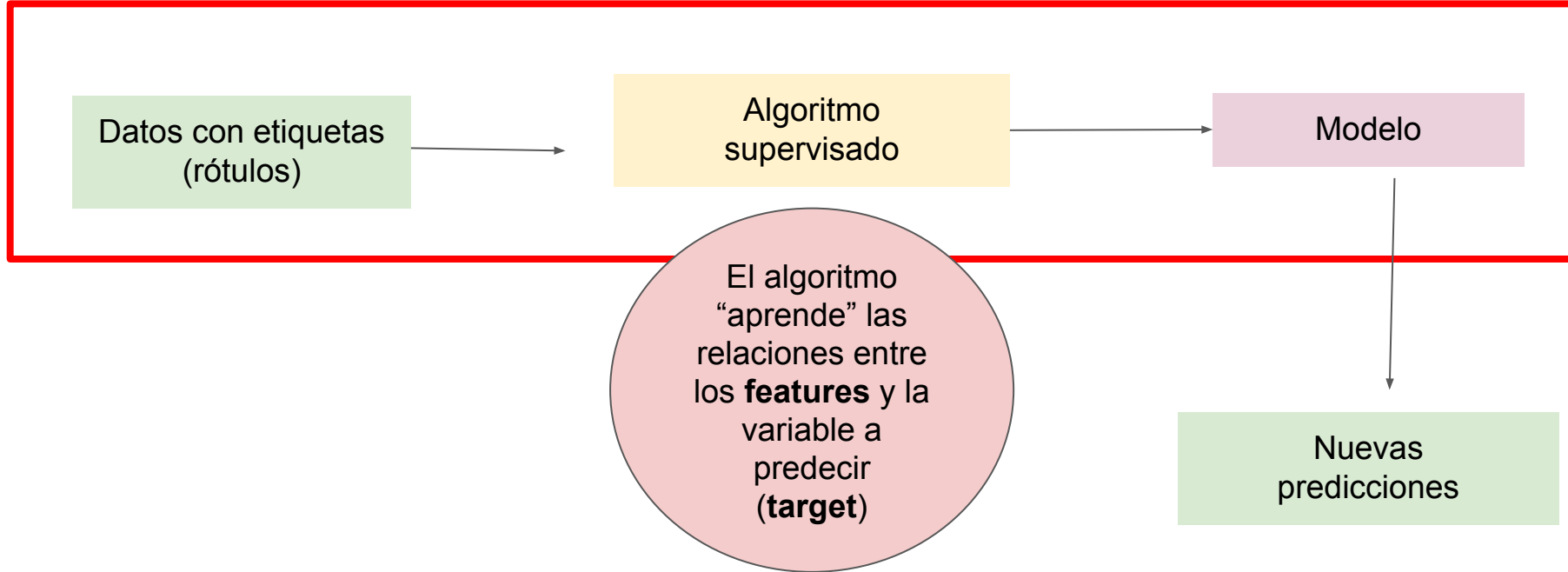
El **aprendizaje supervisado** (Supervised Learning) es una de las categorías principales del aprendizaje automático (Machine Learning). Su característica distintiva es el uso de **datos etiquetados** para entrenar algoritmos.

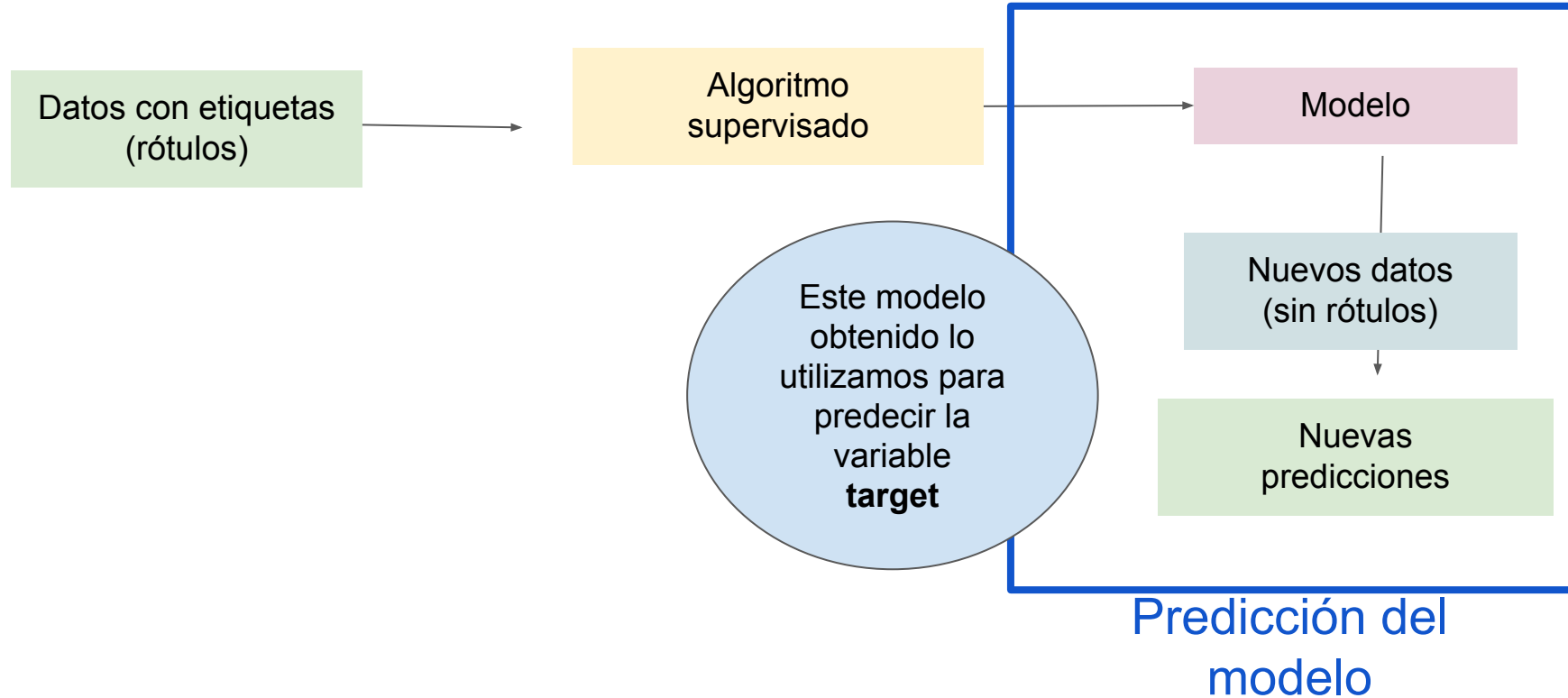
En este modelo, el algoritmo recibe un conjunto de datos de entrenamiento que incluye tanto la **entrada** (input) como el **resultado deseado** (salida o etiqueta). El objetivo es que el modelo aprenda una regla o función matemática que mapee las entradas a las salidas de forma precisa.

El proceso básico:

1. **Entrada de datos:** Se le proporcionan ejemplos al modelo (por ejemplo, fotos de animales).
2. **Etiquetado:** Cada foto viene con una etiqueta (ej. "esto es un gato", "esto es un perro").
3. **Entrenamiento:** El modelo busca patrones (orejas, bigotes, forma de la nariz) que asocien la imagen con su etiqueta.
4. **Predicción:** Una vez entrenado, se le entrega al modelo una foto nueva que nunca ha visto y este debe predecir la etiqueta correcta basándose en lo aprendido.

Entrenamiento del modelo





Clasificación

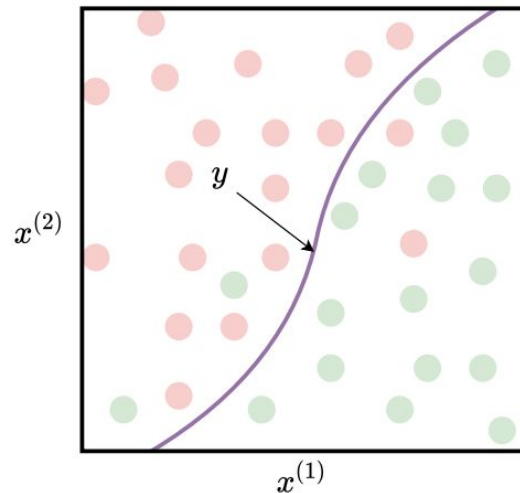
Usado para:

- Filtros de spam (spam vs jam)
- Detección de lenguaje (español, inglés, etc)
- Análisis de sentimientos (positivo / negativo)
- Reconocer dígitos escritos a mano (de 0 a 9).
- Diagnóstico médico (enfermo/sano)
- Clasificación de documentos (política, deportes, tecnología).

Variable a predecir: categoría (o clase)

Clasificación binaria

Clasificación multi-clase

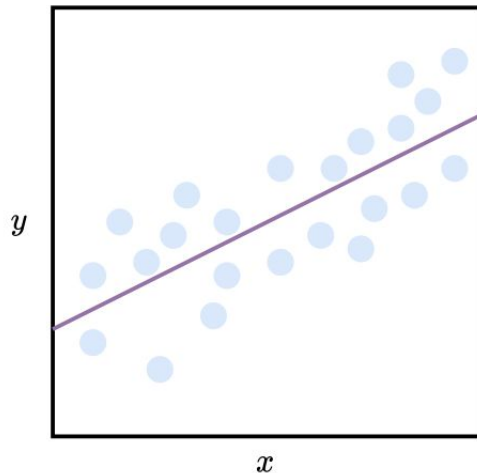


Regresión

Usado para:

- Predicción de precios de casas según variables geográficas, metros cuadrados, etc.
- Establecer relaciones entre ingreso y variables demográficas.
- Predicción de accidentes automovilísticos.
- Predicción de ventas.
- Estimación de consumo energético.
- Predicción de rendimiento de cultivos.
- Predicción de temperatura.

Variable a predecir: numérica



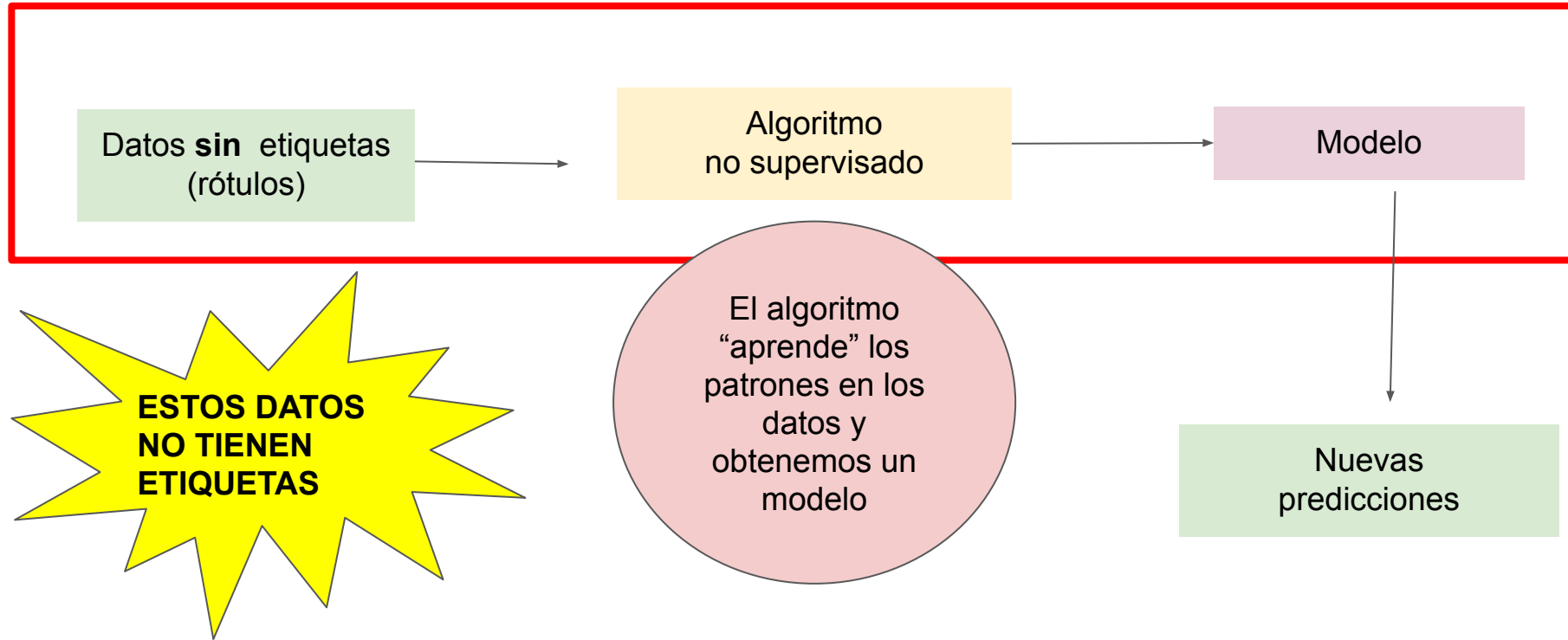
El **aprendizaje no supervisado** (Unsupervised Learning) es una rama de la inteligencia artificial donde el modelo aprende de datos que **no tienen etiquetas** ni respuestas correctas predefinidas.

Imagina que le das a una computadora miles de fotos de diferentes animales, pero **sin decirle qué animal es cada una**. El algoritmo no sabrá que un grupo son "perros" y otro "aves", pero notará que algunas fotos comparten características (como picos y plumas) y las agrupará por similitud.

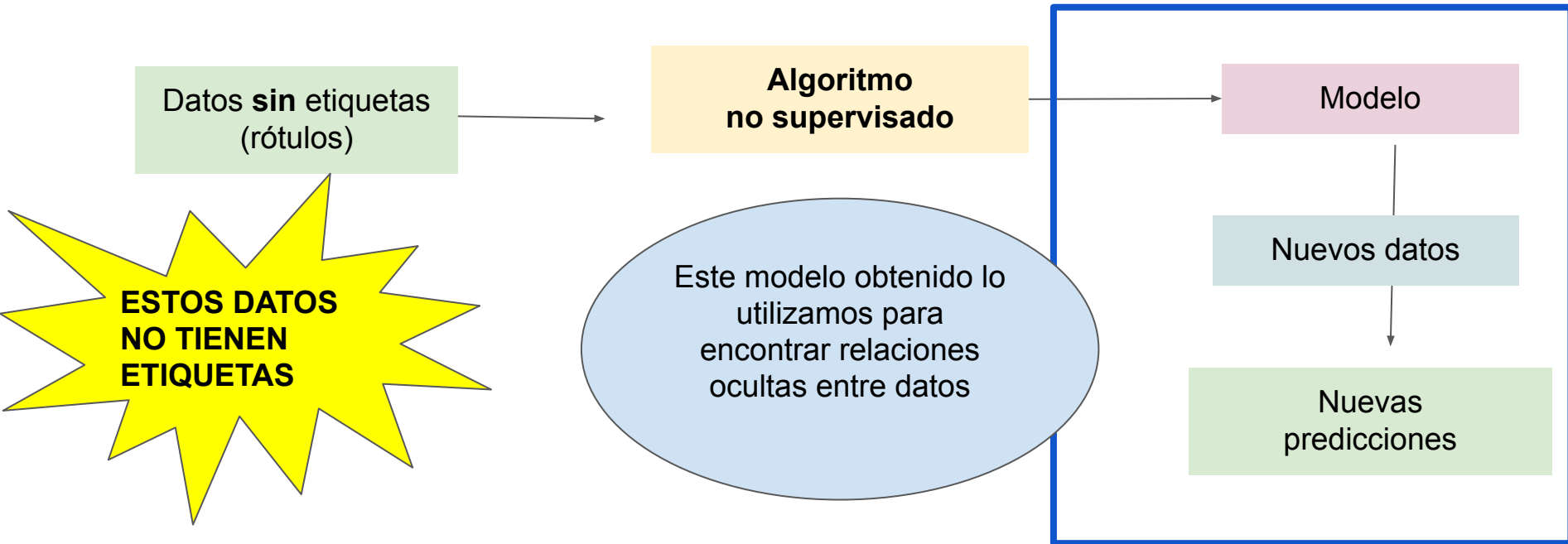
Los objetivos principales son:

1. **Encontrar agrupaciones:** Detectar similitudes naturales en los datos.
2. **Reducir la complejidad:** Simplificar los datos manteniendo la información más importante.
3. **Detectar anomalías:** Identificar datos que no encajan con el resto del patrón.

Entrenamiento del modelo



Aprendizaje no supervisado

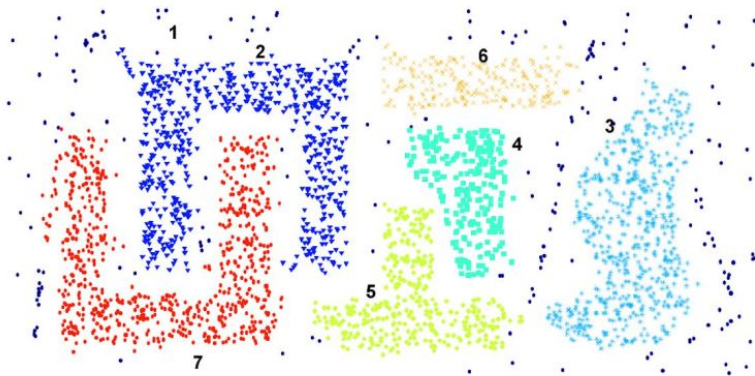


Clusterizado (clustering)

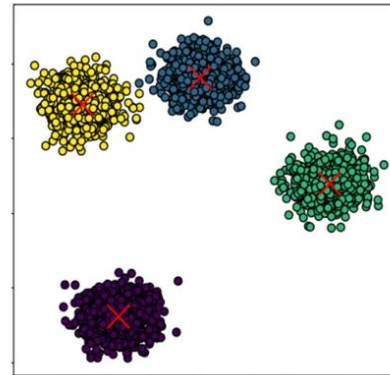
Usado para:

- Segmentación de mercado (identificar tipo de consumidores, consumidores más o menos leales, etc).
- Agrupamiento de productos.
- Agrupamiento geoespacial.
- Optimización de inventario.

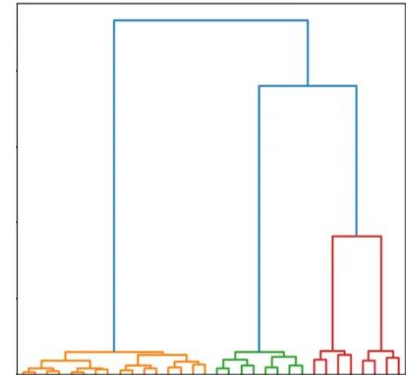
Nuestro objetivo va a ser encontrar posibles agrupamientos en los datos.



K-Means Clustering



Hierarchical Clustering

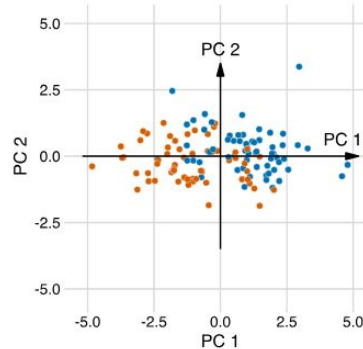
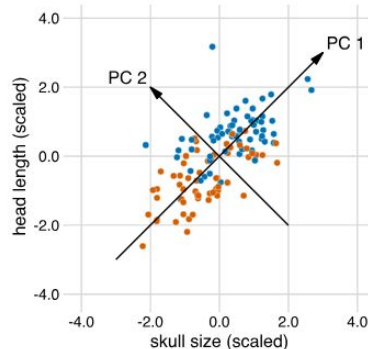
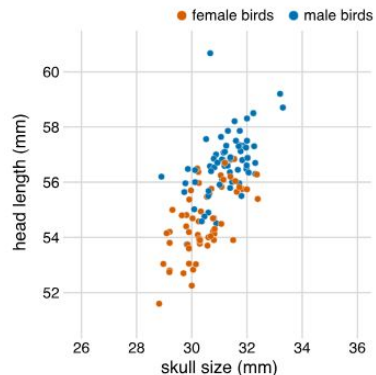


Reducción de dimensiones

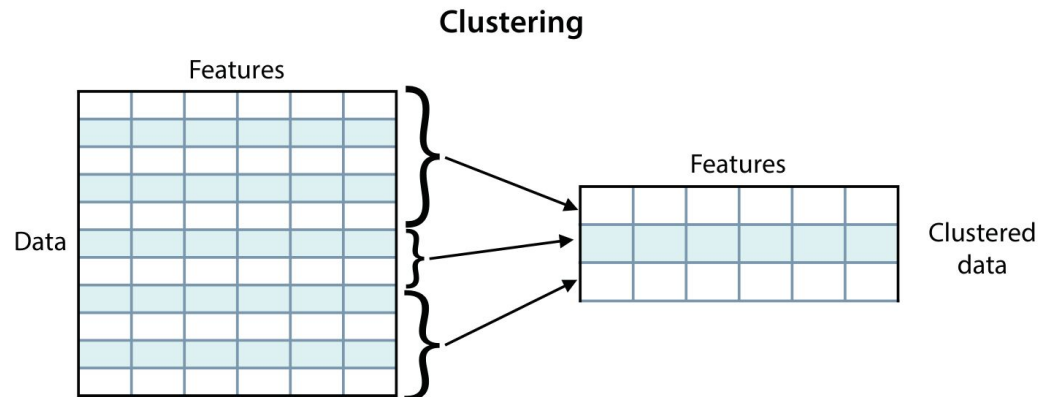
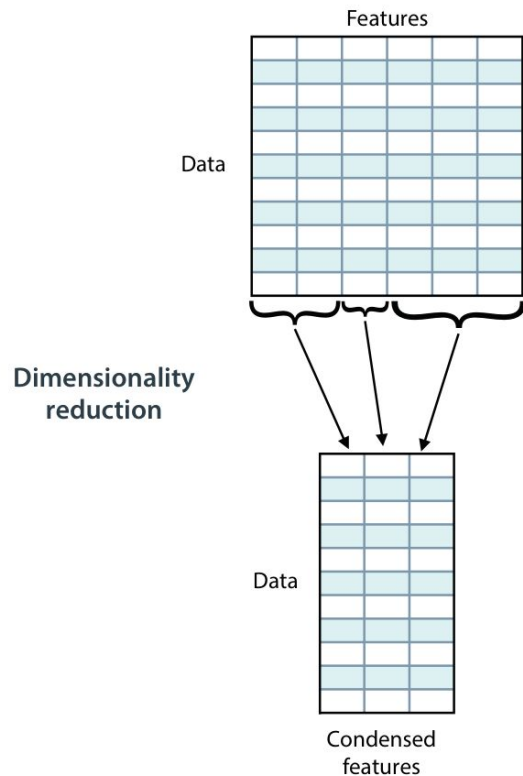
Usado para:

- Modelado de tópicos (análisis de texto).
- Visualización del problema que nos interesa, posibles relaciones entre variables.
- Compresión de imágenes.
- Reducción de características en modelos predictivos.
- Visualización de datos de alta dimensión (UMAP, tSNE).
- Extracción de características en lenguaje natural.

Cuando hacemos reducción de dimensiones, en vez de utilizar las variables que nos proporcionan, vamos a crear variables nuevas que conserven la mayor variabilidad posible.



Clusterizado vs reducción de dimensiones



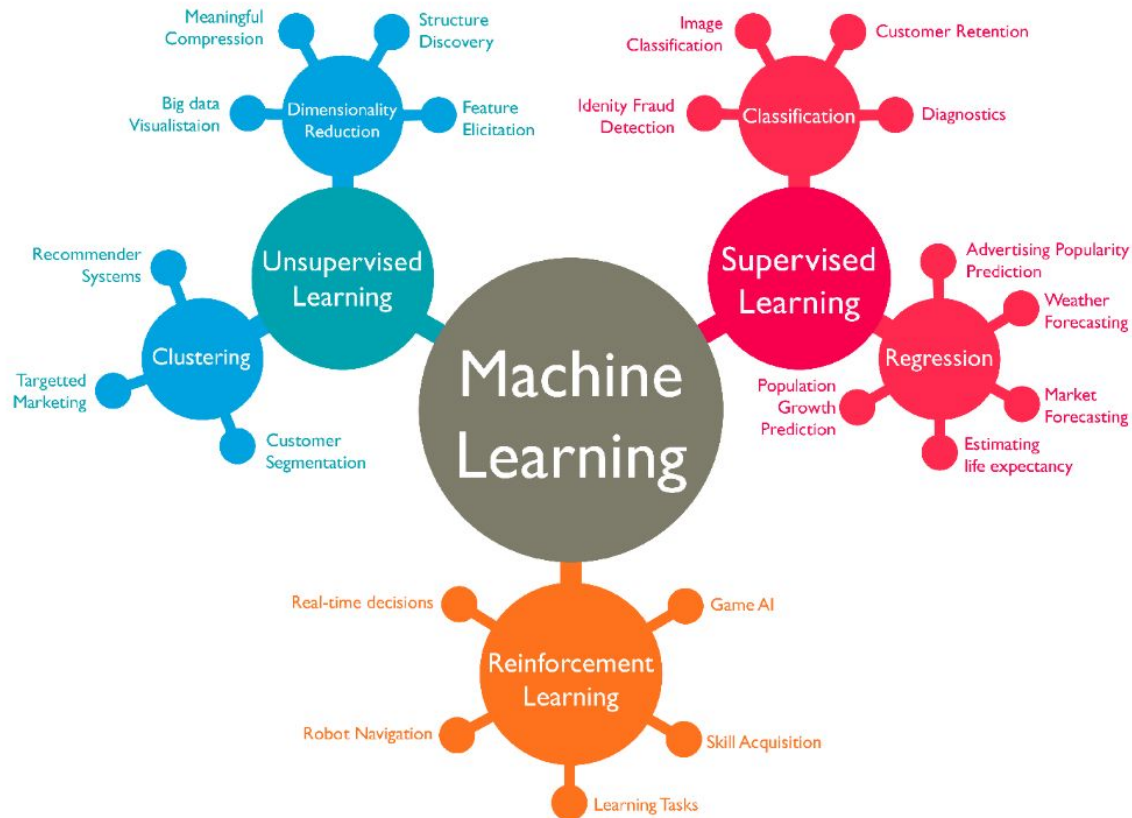
Trabajo a nivel filas

Trabajo a nivel columnas

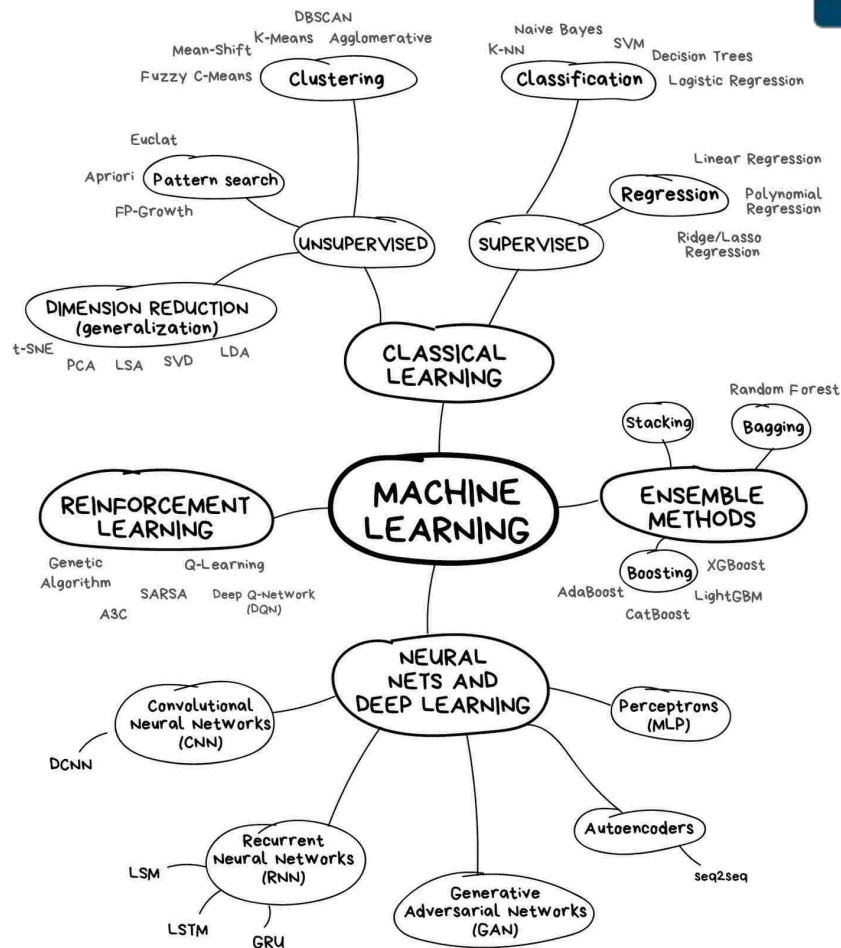
Resumiendo

Análisis de
redes sociales

Embeddings

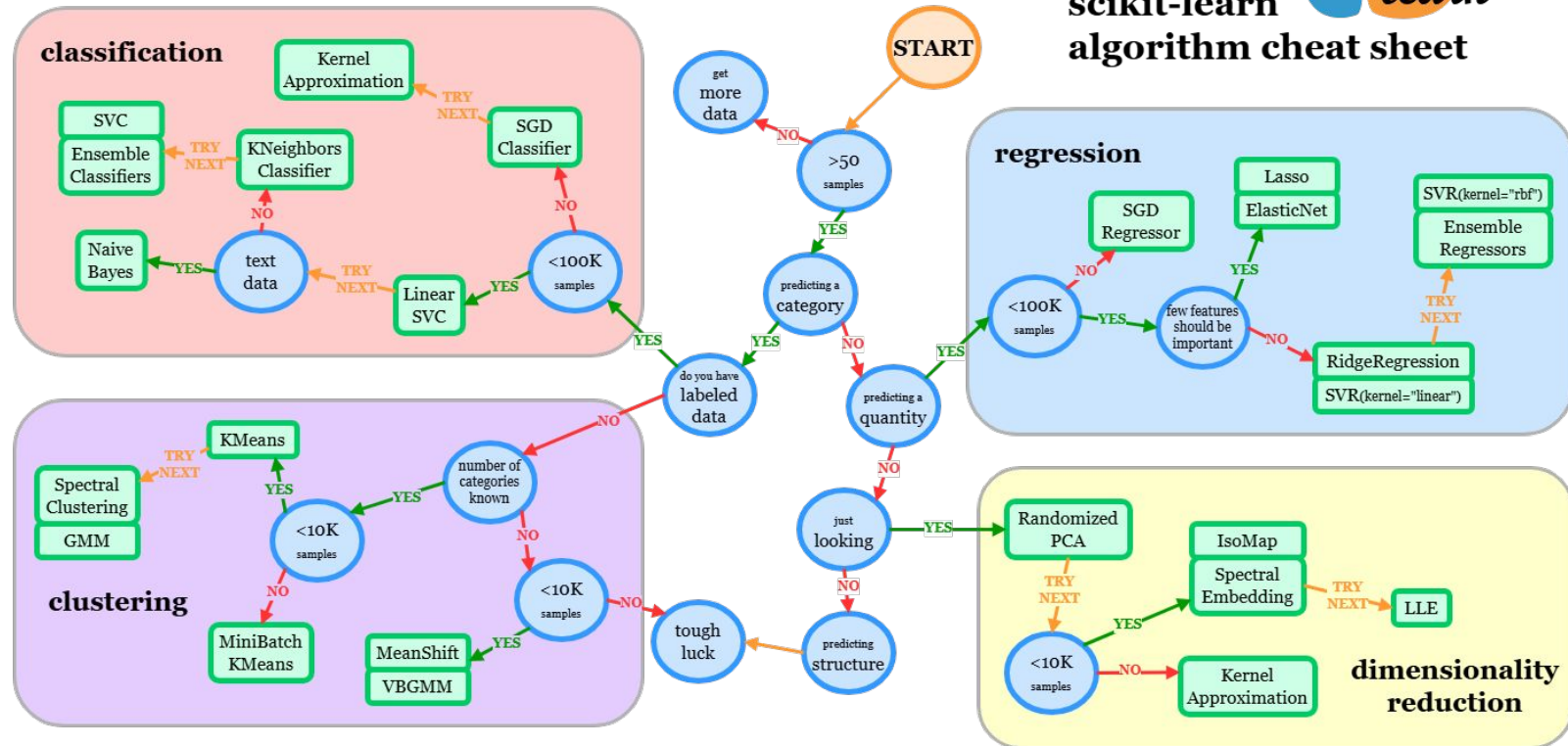


Resumiendo



Resumiendo

scikit-learn **algorithm cheat sheet**



Aprendizaje supervisado vs no supervisado

	Supervisado 	No supervisado 
 Datos de entrada	Con etiquetas	Sin etiquetas
 Proceso de entrenamiento	El modelo recibe los datos de entrada y la etiqueta durante el training	El modelo recibe los datos de entrada sin la etiqueta durante el entrenamiento
 Propósito general	Predecir una salida	Ganar insight de los datos
 Complejidad computacional	Menos complejo	Más complejo
 Tiempo	Consume más tiempo	Consume menos tiempo
 Performance	Más exacto	Menos exacto
 Número de clases	Conocidos de antemano	Desconocidos, el resultado puede ser arbitrario

Modelado Algorítmico

Modelado Estadístico

Las dos culturas

Statistical Science
2001, Vol. 16, No. 3, 199–231

Statistical Modeling: The Two Cultures

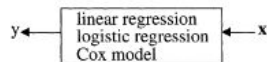
Leo Breiman

Abstract. There are two cultures in the use of statistical modeling to reach conclusions from data. One assumes that the data are generated by a given stochastic data model. The other uses algorithmic models and treats the data mechanism as unknown. The statistical community has been committed to the almost exclusive use of data models. This commitment has led to irrelevant theory, questionable conclusions, and has kept statisticians from working on a large range of interesting current problems. Algorithmic modeling, both in theory and practice, has developed rapidly in fields outside statistics. It can be used both on large complex data sets and as a more accurate and informative alternative to data modeling on smaller data sets. If our goal as a field is to use data to solve problems, then we need to move away from exclusive dependence on data models and adopt a more diverse set of tools.

1. INTRODUCTION

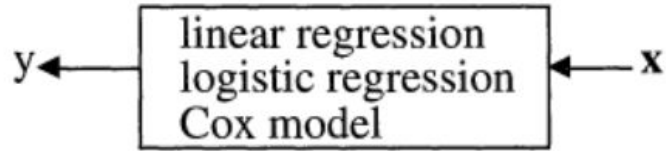
Statistics starts with data. Think of the data as being generated by a black box in which a vector of input variables $\mathbf{x}$ (independent variables) go in one side, and on the other side the response variables $\mathbf{y}$ come out. Inside the black box, nature functions to associate the predictor variables with the response variables, so the picture is like this:

The values of the parameters are estimated from the data and the model then used for information and/or prediction. Thus the black box is filled in like this:

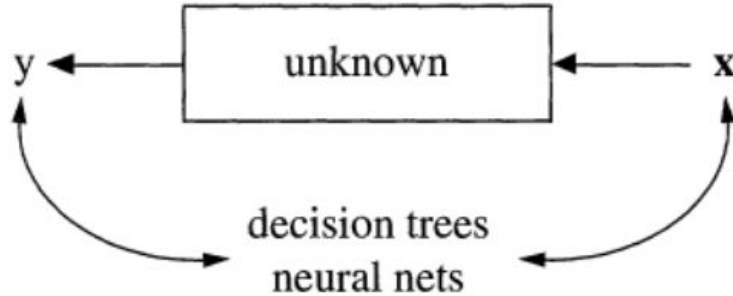


Model validation. Yes—no using goodness-of-fit

Modelado Estadístico vs Modelo Algorítmico



Modelado Estadístico



Modelado Algorítmico

Características Principales

Suposiciones sobre el modelo: En el modelo estadístico, se asume que los datos siguen una distribución específica (por ejemplo, una regresión lineal, una distribución normal, etc.). Esto significa que el analista hace suposiciones claras sobre la relación entre las variables de entrada y la salida.

Parámetros: El objetivo es estimar un conjunto de parámetros dentro de este modelo probabilístico (por ejemplo, los coeficientes de una regresión lineal).

Interpretación: En el enfoque estadístico, se busca comprender la relación entre las variables y hacer inferencias sobre cómo una variable afecta a otra. La **interpretabilidad** es una prioridad.

Ejemplo: La regresión lineal es un ejemplo típico, donde se asume una relación lineal entre las variables independientes y la variable dependiente.

Ventajas:

- Ofrece interpretaciones claras y basadas en principios teóricos.
- Es útil para inferir relaciones causales o comprender los mecanismos que subyacen en los datos.

Desventajas:

- Las suposiciones estrictas del modelo pueden no reflejar la realidad compleja de los datos, lo que lleva a modelos ineficaces cuando las suposiciones son incorrectas.

Características principales:

- **Sin suposiciones fuertes:** A diferencia del enfoque estadístico, los modelos algorítmicos no requieren suposiciones explícitas sobre la forma en que los datos son generados. En lugar de eso, se entrenan modelos complejos (como árboles de decisión, redes neuronales, random forests) directamente a partir de los datos.
- **Predicción:** El objetivo principal es **maximizar la precisión predictiva**. A menudo no se busca una interpretación detallada del modelo o de los parámetros subyacentes.
- **Flexibilidad:** Los algoritmos pueden captar relaciones no lineales y complejas en los datos sin la necesidad de supuestos estrictos sobre las distribuciones de los datos.
- **Ejemplo:** Un modelo como los **random forests** o **deep learning** se basa en el entrenamiento algorítmico de los datos para hacer predicciones precisas sin una estructura explícita sobre cómo las variables están relacionadas.

Ventajas:

- Puede ajustarse mejor a conjuntos de datos complejos y de gran tamaño, y obtener mejores resultados predictivos.
- No requiere suposiciones previas sobre la distribución de los datos.

Desventajas:

- La interpretabilidad puede ser limitada, ya que estos modelos son a menudo "cajas negras". Es decir, aunque predicen bien, no siempre está claro cómo lo hacen.
- Pueden ser propensos al sobreajuste si no se manejan adecuadamente.

Modelado algorítmico vs modelado estadístico

	Modelado Estadístico	Modelado Algorítmico
Supuestos 	Se assume que los datos siguen una distribución específica (ej. lineal, normal).	Sin suposiciones fuertes sobre cómo se generan los datos.
Objetivo Principal 	Interpretabilidad: comprender la relación entre variables y hacer inferencias.	Predicción: maximizar la precisión de los resultados predictivos.
Enfoque 	Énfasis en la función $f(x)$ y en los mecanismos subyacentes de los datos	Énfasis en la predicción y en que el modelo "aprenda" directamente de los datos.
Flexibilidad 	Limitada por supuestos teóricos; puede ser ineficaz si estos son incorrectos.	Alta flexibilidad: capta relaciones no lineales y complejas en grandes volúmenes de datos.

Modelado algorítmico vs modelado estadístico

	Modelado Estadístico	Modelado Algorítmico
Evaluación 	Se basa en estimadores insesgados, robustos y de mínima varianza.	Se basa en la performance predictiva del modelo.
Naturaleza del modelo 	"Caja blanca" : ofrece interpretaciones claras y teóricas.	"Caja negra": la interpretabilidad es limitada; no siempre es claro cómo predice.
Ejemplos típicos 	Regresión lineal, regresión logística, modelo de Cox.	Árboles de decisión, redes neuronales, <i>random forests</i> , <i>deep learning</i> .

1. Definición del problema (problem framing) y recolección de datos.
2. Preparación y limpieza de datos (Pre-processing)
3. División del conjunto de datos.
4. Selección y Entrenamiento del modelo
5. Definición de una métrica de éxito
6. Evaluación de desempeño
7. Ajuste de hiperparámetros (tunning)
8. Despliegue (deployment) y monitoreo*

* solo en entornos de producción

1. Definición del problema (Problem Framing) y recolección

Antes de tocar el código, debes traducir una necesidad de negocio a un problema de ML.

- **Framing:** ¿Es un problema de clasificación (¿es A o B?), de regresión (¿cuánto valdrá?) o de agrupamiento?
- **Recolección:** Identificar las fuentes de datos (bases de datos SQL, logs de servidores, APIs externas, encuestas). Sin datos de calidad, el modelo no podrá aprender (el famoso concepto *Garbage In, Garbage Out*).

2. Preparación y limpieza (Pre-processing)

Es la etapa que más tiempo consume. Los datos del mundo real son "sucios".

- **Limpieza:** Manejar valores nulos (borrarlos o inventar uno basado en el promedio) y eliminar duplicados.
- **Transformación:** Convertir texto en números (Label Encoding) y escalar valores (por ejemplo, normalizar para que la "edad" de 0-100 y el "salario" de 0-1.000.000 tengan el mismo peso visual para el algoritmo).

3. División del conjunto de datos (Data Splitting)

Para saber si tu modelo realmente aprendió o solo memorizó, debes dividir tus datos:

- **Training Set (Entrenamiento):** Generalmente el 70-80% de los datos. Se usan para "enseñar" al modelo.
- **Test Set (Prueba):** El 20-30% restante. El modelo nunca los ha visto. Se usan para simular cómo se comportará en el mundo real.

4. Selección y Entrenamiento del modelo

Aquí eliges el algoritmo según el problema:

- **Selección:** ¿Usarás una Regresión Lineal (simple), un Bosque Aleatorio (Random Forest) o una Red Neuronal Profunda (Deep Learning)?
- **Entrenamiento:** Es el proceso donde el algoritmo busca patrones en los datos de entrenamiento para ajustar sus parámetros internos y minimizar el error.

5. Definición de una métrica de éxito

¿Cómo sabemos si el modelo es "bueno"?

- **Exactitud (Accuracy):** Porcentaje de aciertos totales.
- **Precisión y Recall:** Vitales si, por ejemplo, estás detectando cáncer; donde un "falso negativo" es mucho más grave que un "falso positivo".
- **RMSE:** Se usa en predicciones numéricas (como el precio de una casa) para saber qué tan lejos estuvimos del valor real en promedio.

6. Evaluación de desempeño

Se pone a prueba el modelo con el **Test Set**. Si el modelo funciona excelente con los datos de entrenamiento pero pésimo con los de prueba, sufres de **Overfitting** (sobreajuste): el modelo memorizó los datos en lugar de aprender a generalizar.

7. Ajuste de hiperparámetros (Tuning)

Los modelos tienen "perillas" externas llamadas hiperparámetros (como la velocidad de aprendizaje o la profundidad de un árbol).

- **Optimización:** Se prueban diferentes combinaciones de estas perillas para extraer el máximo rendimiento posible del modelo seleccionado.

8. Despliegue (Deployment) y monitoreo

- **Despliegue:** El modelo se convierte en una API o se integra en una aplicación (por ejemplo, el botón de "recomendar" de Netflix).
- **Monitoreo:** El mundo cambia. Si los usuarios cambian de comportamiento (por ejemplo, por una pandemia), el modelo dejará de ser preciso. Esto se llama *Data Drift* y requiere volver al paso 1 para reentrenar.

Proceso del Aprendizaje Automático

El proceso de modelado no es un evento único, sino un ciclo iterativo y multifacético que involucra varias etapas críticas. El objetivo principal es construir un modelo que no solo se desempeñe bien con los datos que ya ha "visto" (los datos de entrenamiento), sino que también sea capaz de **generalizar** su conocimiento para hacer predicciones precisas sobre datos nuevos y no vistos.

- División de los datos: Train / test split
- Complejidad / Interpretabilidad del modelo
- Parámetros / hiperparámetros
- Error de entrenamiento / error de generalización
- Underfitting vs overfitting
- Métricas

División de datos

Generalidades

Datos ordenados

attributes

Country	Population	Region	GDP
France	67M	Europe	2.6T
Germany	83M	Europe	3.7T
...	...	...	...
China	1386M	Asia	12.2T

examples

Country	Population	Region	GDP
France	67M	Europe	2.6T
Germany	83M	Europe	3.7T
...	...	...	...
China	1386M	Asia	12.2T

Figure 1.3: Tidy data: examples are rows and attributes are columns.

Datos ordenados

observación o muestra

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

features / variables: columnas

target:
variable a
predecir

- **Muestra, punto, observación, instancia** se refiere a una unidad de análisis.
- **Set de entrenamiento** son los datos utilizados para el modelado.
- **Set de prueba** son los datos utilizados para medir el desempeño del modelo, entre un conjunto de candidatos.
- **Atributos, predictores, features, variables independientes o descriptores** son los datos de entrada para la ecuación de predicción.
- **Salida, variable dependiente, variable respuesta, clase, o "target"** es la cantidad/clase a ser predicha.

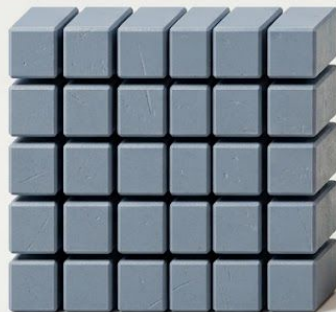
Datos ordenados

Scalar: One number
(e.g. heart rate)

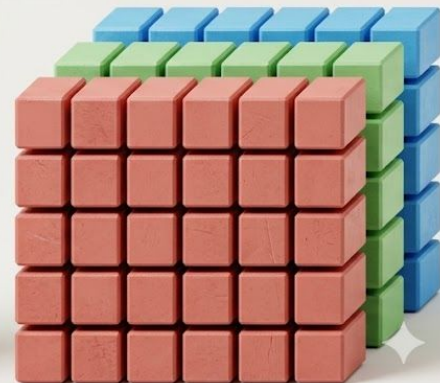
Vector: a row of numbers
(e.g. heart rate sequence)



Matrix: a rectangle of numbers
(e.g. MRI image, grayscale images)



Tensor: a cube of numbers
(e.g. **R****G****B** color image)



Tabular Data

columns = attributes for those observations

Player	Minutes	Points	Rebounds	Assists
A	41	20	6	5
B	30	29	7	6
C	22	7	7	2
D	26	3	3	9
E	20	19	8	0
F	9	6	14	14
G	14	22	8	3
I	22	36	0	9
J	34	8	1	3

Rows = observations

Niveles de complejidad

Aspecto	Escalar	Vector	Matriz	Tensor
Dimensionalidad	0D	1D	2D	$\geq 3D$
Representación	Valor numérico único	Matriz ordenada de valores	Cuadrícula bidimensional de valores	Matriz multidimensional de valores
Uso	Representar cantidades básicas	Representar características o direcciones	Organizar los datos en forma de tabla	Representar estructuras de datos complejas
Ejemplos	Métrica de error, probabilidad	Vector de características, gradiente	Matriz de datos, matriz de pesos	Tensor de imagen, tensor de vídeo

Niveles de complejidad

Aspecto	Escalar	Vector	Matriz	Tensor
Manipulación	Operaciones aritméticas	Operaciones de álgebra lineal	Operaciones matriciales, transformaciones	Operaciones tensoriales en aprendizaje profundo
Representación de datos	Punto en el espacio	Dirección y magnitud	Filas y columnas	Relaciones multidimensionales
Aplicaciones	Cálculos básicos, estadísticas	Características de ML, incrustaciones	Manipulación y análisis de datos	Aprendizaje profundo, PNL, visión por computadora
Notación	Letras minúsculas (p. ej., x)	Minúsculas en negrita (p. ej., v)	Mayúsculas en negrita (p. ej., M)	Mayúsculas indexadas (p. ej., T_{ijk})

Train - test split

Opción 1: HOLDOUT

El método **Holdout** es la técnica más sencilla y directa para dividir un *dataset*. Consiste en separar los datos disponibles en dos subconjuntos principales: un conjunto de **entrenamiento (Train)** y un conjunto de **testeo (Test)**.



Objetivo de train-test split



El objetivo es obtener una **estimación imparcial** del rendimiento del modelo. Al separar un conjunto de prueba (*test set*) que el modelo jamás ve durante el entrenamiento, simulamos cómo se comportará cuando reciba datos nuevos de clientes, sensores o usuarios reales en el futuro.

Esto sirve específicamente para detectar el **sobreajuste (overfitting)**: si el modelo tiene un 99% de precisión en el entrenamiento pero un 70% en el test, sabes que el modelo no aprendió, sino que memorizó.

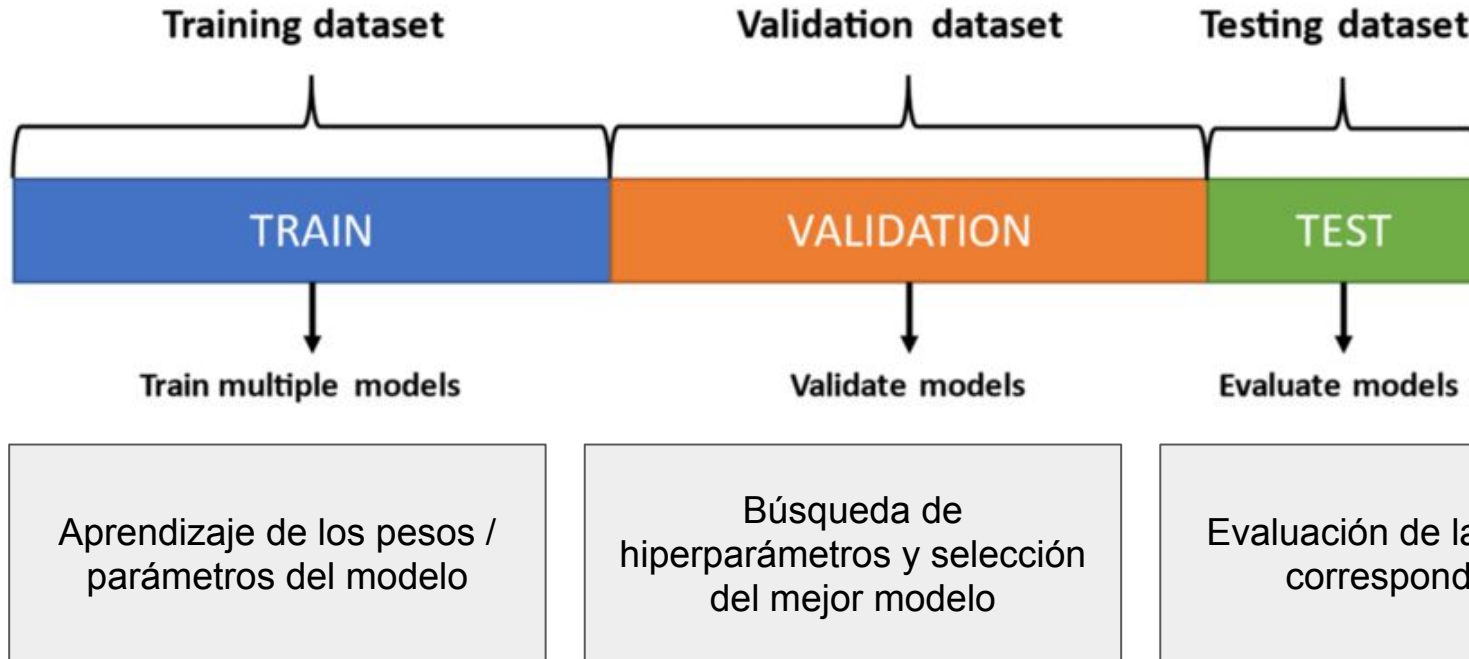
Cuando utilizar esta división de datos

El holdout se prefiere en situaciones específicas:

- **Datasets Masivos:** Si tienes millones de registros, hacer validación cruzada es innecesariamente lento. Una sola partición es suficiente para ser representativa.
- **Modelos Costosos:** Si entrenar tu modelo (como una red neuronal profunda) tarda días, no puedes permitirte entrenarlo 5 o 10 veces como exige la validación cruzada.
- **Prototipado Rápido:** Cuando necesitas una métrica de referencia rápida para saber si un algoritmo tiene potencial antes de invertir más tiempo.

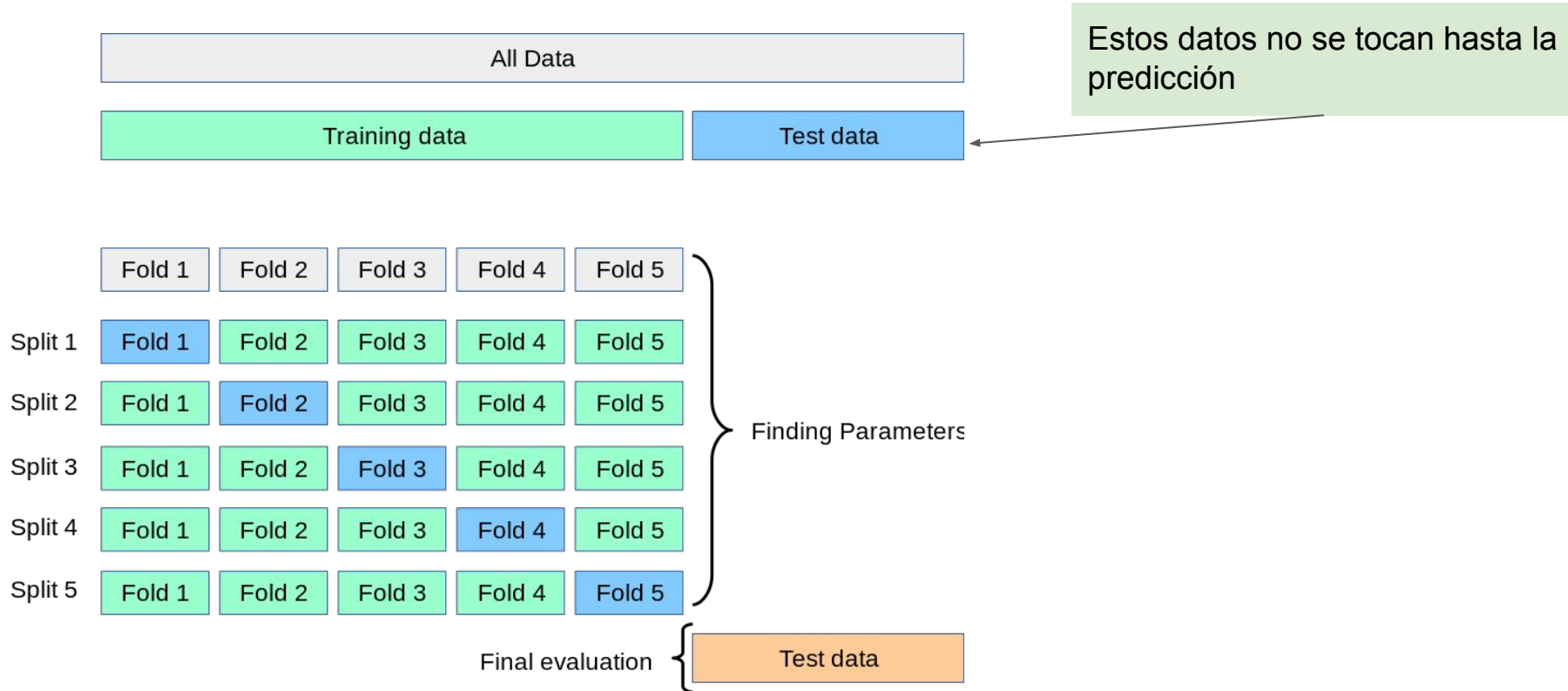
Train - validation - test split

Opción 2: Train - validation - test split



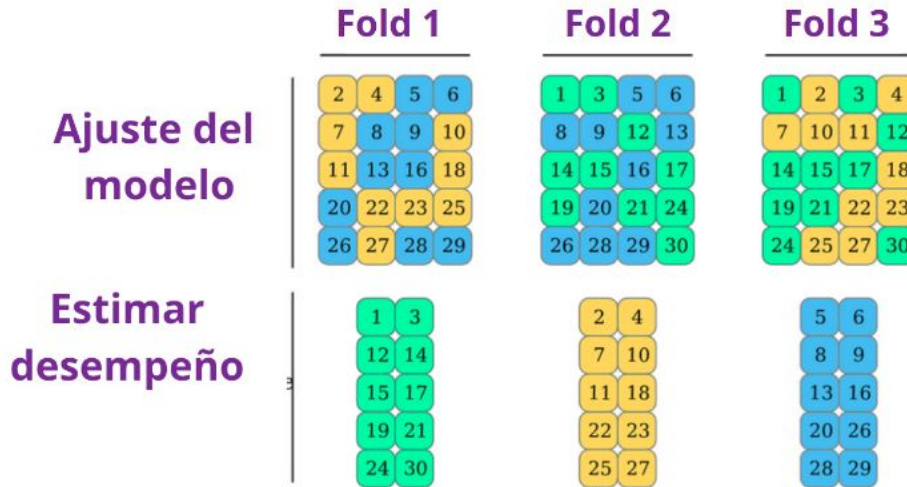
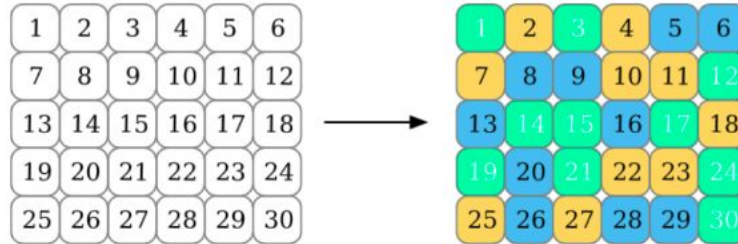
Validación cruzada

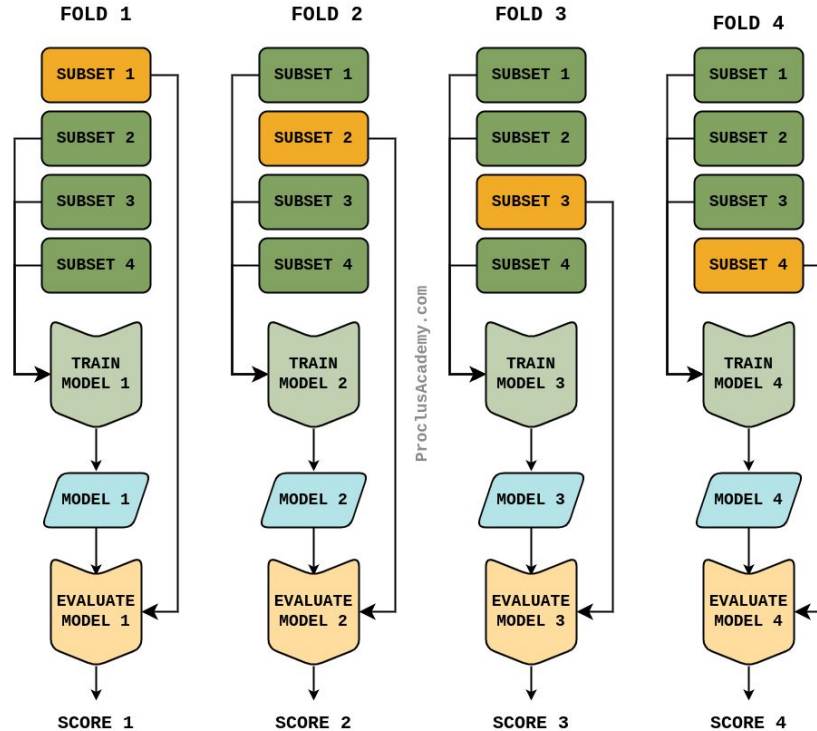
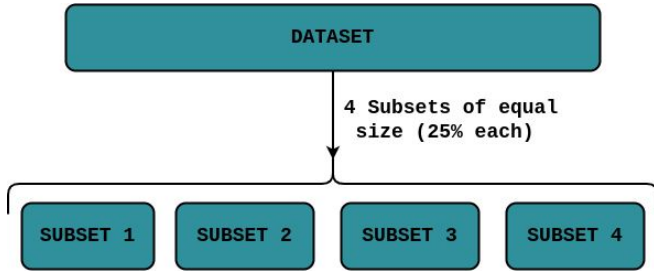
Opción 3: Cross - validation



Validación cruzada

Train data





1. **Evitar el overfitting**: Si se entrena y se prueba un modelo con el mismo conjunto de datos, es probable que el modelo simplemente "memorice" los datos en lugar de aprender los patrones. La validación cruzada divide los datos repetidamente para asegurar que el modelo se evalúe con información que no usó para aprender, dándote una imagen real de su precisión.
2. **Aprovechar al máximo los datos disponibles**: Cuando se tiene pocos datos, separar un 20% para "test" puede dejarte con muy poca información para entrenar. La validación cruzada (especialmente la de **K-fold**) permite que cada dato sea utilizado tanto para entrenar como para validar en diferentes momentos.
3. **Reducir el sesgo en la evaluación**: Si se hace una sola partición aleatoria (entrenamiento/prueba), podrías tener "mala suerte" y que el grupo de prueba sea demasiado fácil o demasiado difícil. Al promediar los resultados de varias iteraciones, obtienes una métrica de desempeño mucho más estable y confiable.

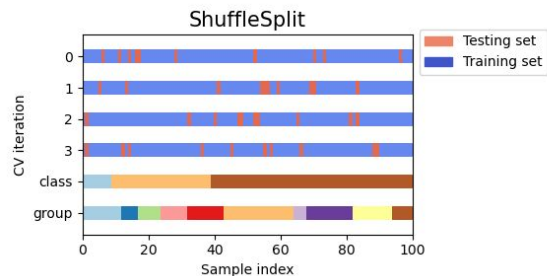
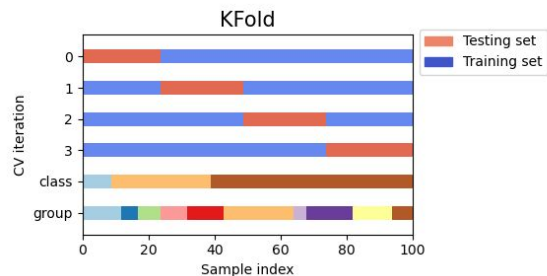
- **Selección de hiperparámetros:** Ayuda a decidir qué configuración de un modelo (por ejemplo, la profundidad de un árbol de decisión, cantidad de hojas en un árbol, C y gamma en SVM, etc) funciona mejor.
- **Comparación de modelos:** Permite saber si un algoritmo de Bosques Aleatorios es realmente mejor que una Regresión Logística para un problema específico.

Otros tipos de validación cruzada

Para datos independientes e idénticamente distribuidos (i.i.d.)

Son los más comunes cuando no hay una relación temporal o de grupo entre las muestras:

- **K-Fold:** Divide los datos en K grupos (pliegues). Entrena en $K-1$ y valida en el restante. Se repite K veces.
- **Repeated K-Fold:** Repite el proceso de K-Fold n veces con diferentes divisiones aleatorias en cada repetición para obtener una estimación más robusta.
- **Leave One Out (LOO):** Cada muestra se utiliza una vez como set de prueba, mientras que las demás son el set de entrenamiento. Es exhaustivo pero costoso en términos de computación.
- **Shuffle & Split:** Genera un número determinado de divisiones de entrenamiento/prueba mezclando los datos aleatoriamente en cada iteración. A diferencia de K-Fold, el tamaño de las iteraciones no depende del número de pliegues.

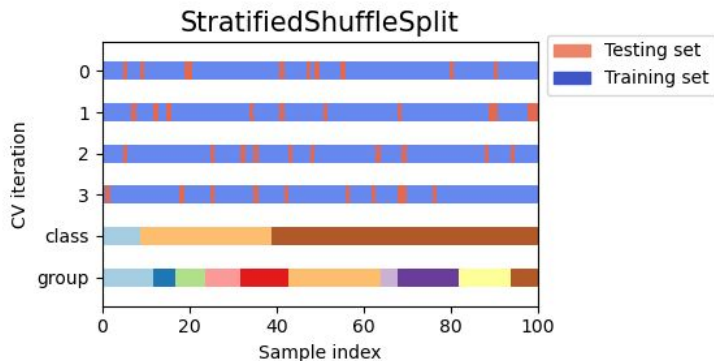
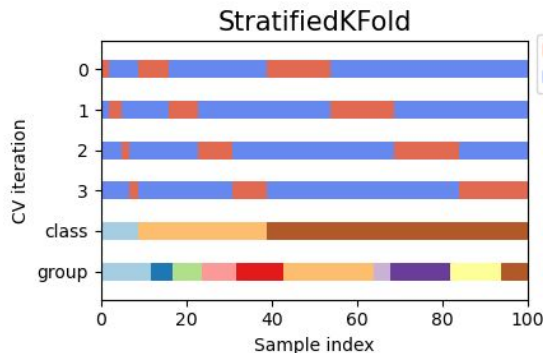


Otros tipos de validación cruzada

Para datos con clases desequilibradas (Estratificados)

Se usan en problemas de clasificación para asegurar que cada pliegue mantenga la misma proporción de clases que el dataset original:

- **Stratified K-Fold:** Una variante de K-Fold que preserva el porcentaje de muestras de cada clase en cada pliegue.
- **Stratified Shuffle Split:** Combina la mezcla aleatoria con la preservación de la proporción de clases.

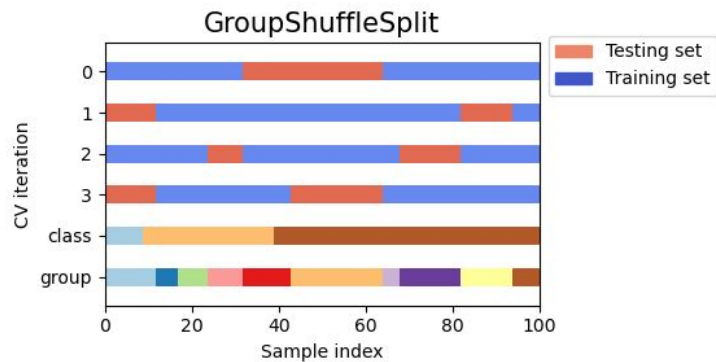
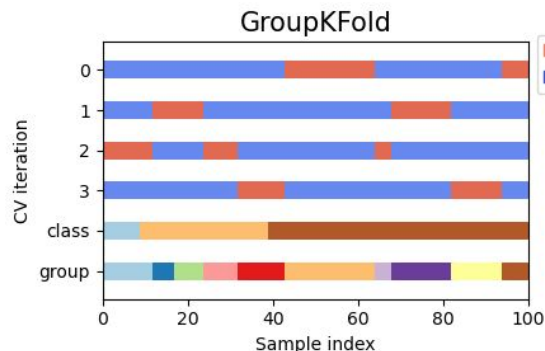


Otros tipos de validación cruzada

Para datos agrupados (Grouped Data)

Se utilizan cuando tienes muestras que dependen de un grupo específico (por ejemplo, múltiples mediciones de un mismo paciente) y quieres evitar que datos del mismo grupo estén en entrenamiento y prueba al mismo tiempo:

- **Group K-Fold:** Asegura que el mismo grupo no aparezca en dos pliegues distintos.
- **Leave One Group Out:** Deja fuera un grupo completo para validación en cada iteración.
- **Group Shuffle Split:** Mezcla aleatoria que mantiene la integridad de los grupos.

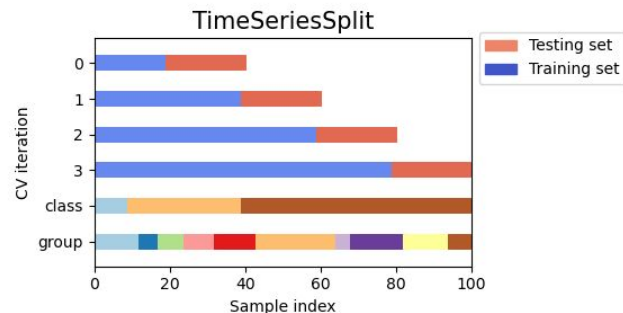


Otros tipos de validación cruzada

Para series temporales (Time Series)

Cuando el orden cronológico importa y no puedes usar datos del "futuro" para predecir el "pasado":

- **Time Series Split:** Una variante de K-Fold donde en cada división el set de entrenamiento es mayor y el set de prueba es siempre posterior en el tiempo a los datos de entrenamiento (ventana expansiva).

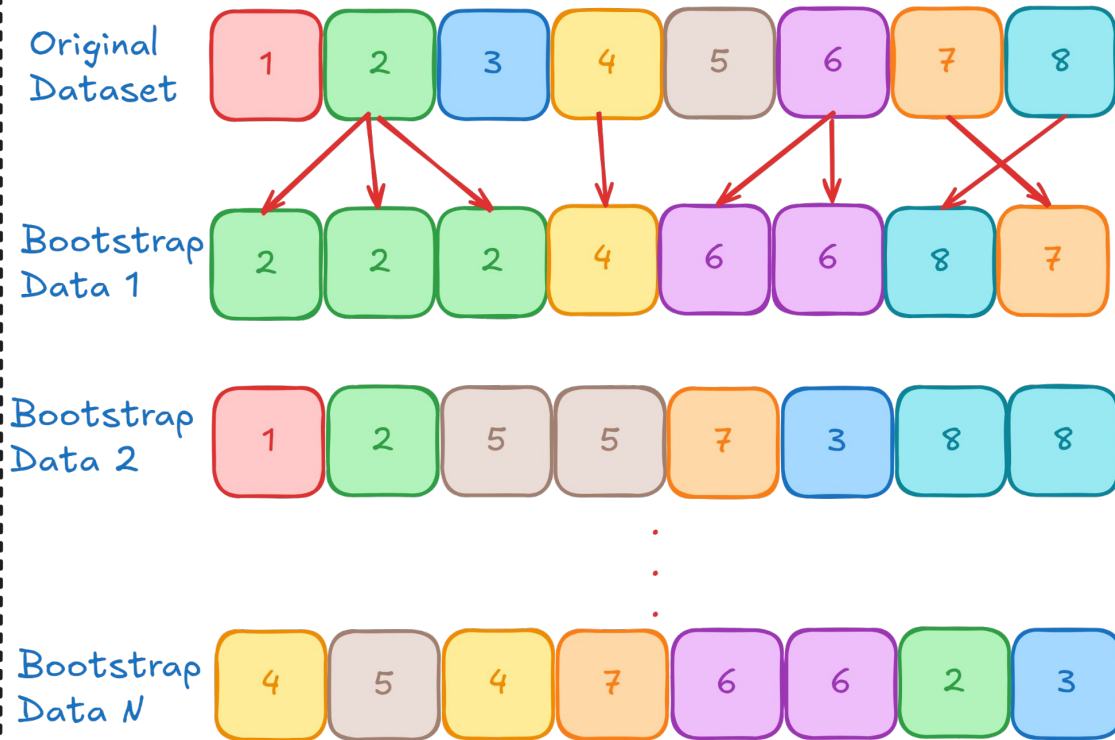


Opción 4 : Bootstrapping

	Bootstrap iter 1	Bootstrap iter 2	Bootstrap iter 3																																																																																										
Ajuste del modelo	<table><tr><td>1</td><td>1</td><td>4</td><td>7</td><td>8</td><td>8</td></tr><tr><td>10</td><td>13</td><td>13</td><td>13</td><td>14</td><td>15</td></tr><tr><td>16</td><td>16</td><td>16</td><td>17</td><td>19</td><td>19</td></tr><tr><td>21</td><td>22</td><td>23</td><td>23</td><td>24</td><td>23</td></tr><tr><td>25</td><td>25</td><td>25</td><td>27</td><td>28</td><td>29</td></tr></table>	1	1	4	7	8	8	10	13	13	13	14	15	16	16	16	17	19	19	21	22	23	23	24	23	25	25	25	27	28	29	<table><tr><td>2</td><td>2</td><td>3</td><td>3</td><td>3</td><td>4</td></tr><tr><td>4</td><td>4</td><td>6</td><td>6</td><td>7</td><td>10</td></tr><tr><td>11</td><td>12</td><td>12</td><td>14</td><td>14</td><td>15</td></tr><tr><td>17</td><td>17</td><td>18</td><td>21</td><td>22</td><td>22</td></tr><tr><td>23</td><td>23</td><td>28</td><td>27</td><td>28</td><td>30</td></tr></table>	2	2	3	3	3	4	4	4	6	6	7	10	11	12	12	14	14	15	17	17	18	21	22	22	23	23	28	27	28	30	<table><tr><td>2</td><td>2</td><td>3</td><td>3</td><td>4</td><td>5</td></tr><tr><td>5</td><td>5</td><td>6</td><td>7</td><td>10</td><td>11</td></tr><tr><td>12</td><td>15</td><td>16</td><td>18</td><td>18</td><td>19</td></tr><tr><td>19</td><td>20</td><td>20</td><td>20</td><td>21</td><td>21</td></tr><tr><td>21</td><td>21</td><td>22</td><td>22</td><td>29</td><td>30</td></tr></table>	2	2	3	3	4	5	5	5	6	7	10	11	12	15	16	18	18	19	19	20	20	20	21	21	21	21	22	22	29	30
1	1	4	7	8	8																																																																																								
10	13	13	13	14	15																																																																																								
16	16	16	17	19	19																																																																																								
21	22	23	23	24	23																																																																																								
25	25	25	27	28	29																																																																																								
2	2	3	3	3	4																																																																																								
4	4	6	6	7	10																																																																																								
11	12	12	14	14	15																																																																																								
17	17	18	21	22	22																																																																																								
23	23	28	27	28	30																																																																																								
2	2	3	3	4	5																																																																																								
5	5	6	7	10	11																																																																																								
12	15	16	18	18	19																																																																																								
19	20	20	20	21	21																																																																																								
21	21	22	22	29	30																																																																																								
Estimar desempeño	<table><tr><td>2</td><td>3</td><td>5</td><td>6</td><td>9</td><td>11</td></tr><tr><td>12</td><td>18</td><td>20</td><td>24</td><td>26</td><td>28</td></tr><tr><td></td><td></td><td>30</td><td></td><td></td><td></td></tr></table>	2	3	5	6	9	11	12	18	20	24	26	28			30				<table><tr><td>1</td><td>5</td><td>8</td><td>9</td><td>13</td><td>16</td></tr><tr><td>19</td><td>20</td><td>24</td><td>26</td><td>29</td><td></td></tr></table>	1	5	8	9	13	16	19	20	24	26	29		<table><tr><td>1</td><td>8</td><td>9</td><td>13</td><td>14</td><td>17</td></tr><tr><td>23</td><td>24</td><td>25</td><td>26</td><td>27</td><td>28</td></tr></table>	1	8	9	13	14	17	23	24	25	26	27	28																																																
2	3	5	6	9	11																																																																																								
12	18	20	24	26	28																																																																																								
		30																																																																																											
1	5	8	9	13	16																																																																																								
19	20	24	26	29																																																																																									
1	8	9	13	14	17																																																																																								
23	24	25	26	27	28																																																																																								

Implica repetir
el mismo
elemento
varias veces

Traditional Bootstrap



Implica repetir
el mismo
elemento
varias veces

Objetivo del bootstrap

El objetivo de usar Bootstrap para dividir datos es **estimar qué tan sensible es el rendimiento del modelo a los cambios en el conjunto de entrenamiento.**

A diferencia de una división train-test (donde divides 80/20 una sola vez), el Bootstrap genera múltiples escenarios de entrenamiento. Si el modelo da resultados muy distintos en cada muestra Bootstrap, significa que el modelo es **inestable** o que los datos tienen mucho ruido.

1. Bagging (bootstrap aggregation). Se desarrollará en modelos de ensamble.
2. Estimación del “out of bag error”. Se desarrollará en modelos de ensamble.
3. Comparación de modelos cuando existen pocos ejemplos. Cuando tienes tan pocos datos que una división *Train/Test/Validation* tradicional dejaría los conjuntos casi vacíos, el Bootstrap permite:
 - Generar múltiples conjuntos de entrenamiento.
 - Obtener una distribución de la precisión del modelo (no solo un número, sino un rango).

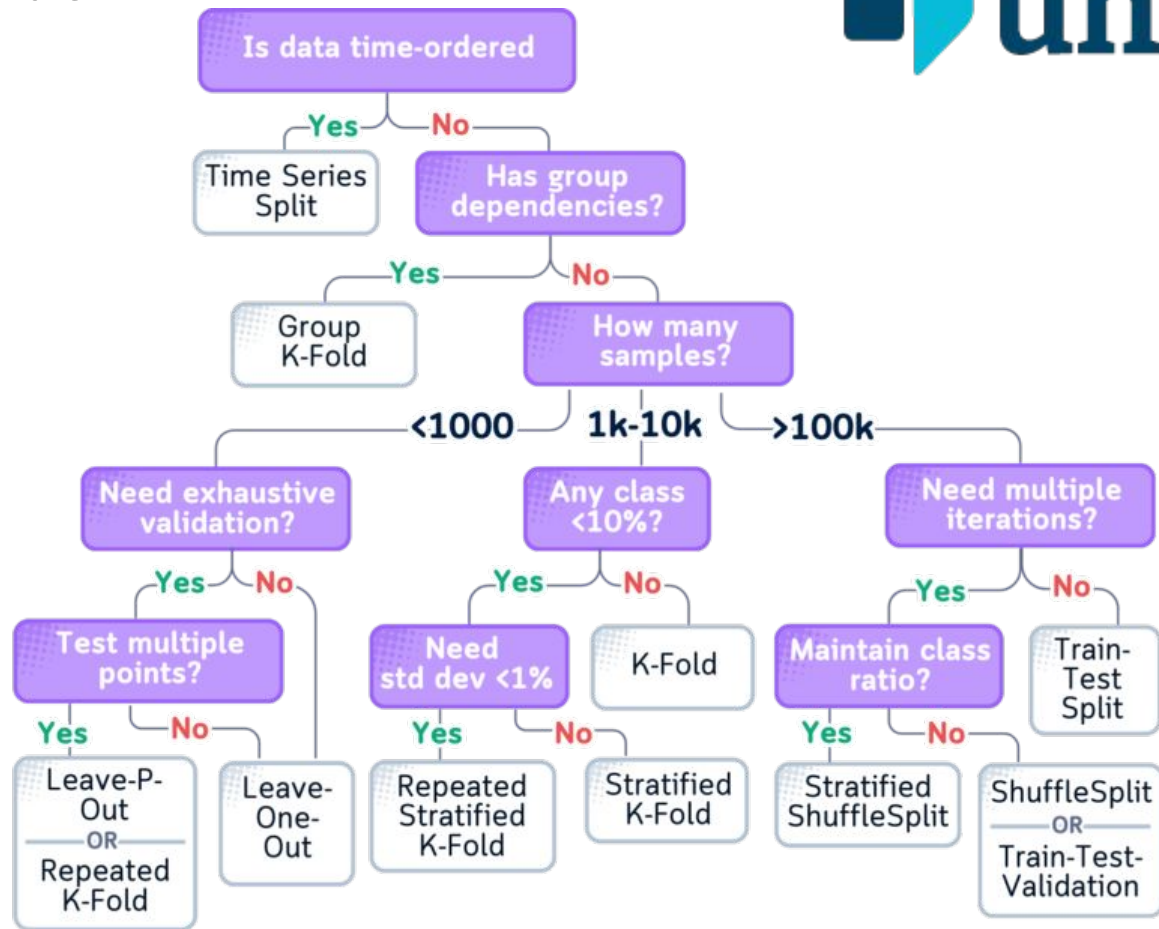
Tanto la validación cruzada como el bootstrap son técnicas de resampléo (resampling).

La diferencia radica en que la validación cruzada no es con reemplazo mientras que el bootstrap sí lo es.

Resumiendo

	Uso Principal	Ventajas	Desventajas
Train-Test Split	Evaluación rápida y final del rendimiento.	Muy rápido y computacionalmente económico.	Alto riesgo de sesgo si la división "aleatoria" no es representativa.
Train-Validation-Test Split	Ajuste de hiperparámetros (Tuning) y selección de modelos.	Evita el "leakage" (fuga de información) del set de prueba durante el ajuste.	Requiere muchos datos para que cada set sea significativo.
Validación Cruzada (K-Fold CV)	Estimación robusta del error en datasets medianos/pequeños .	Usa todos los datos para entrenar y probar. Reduce mucho la varianza.	Costoso computacionalmente (entrenas el modelo \$K\$ veces).
Bootstrap	Estimación de incertidumbre y datasets muy pequeños .	Provee intervalos de confianza y funciona bien con pocos datos.	Puede llevar al sobreajuste (overfitting) por la repetición de datos.

Resumiendo



Complejidad vs interpretabilidad del modelo

Interpretabilidad vs Flexibilidad

Modelos Interpretables (Simples): Son modelos con baja flexibilidad. Su ventaja es que ofrecen una comprensión clara de la relación entre las variables de entrada y la salida (como una regresión lineal o logística). Priorizan la inferencia y la capacidad de explicar el "porqué" de un resultado.

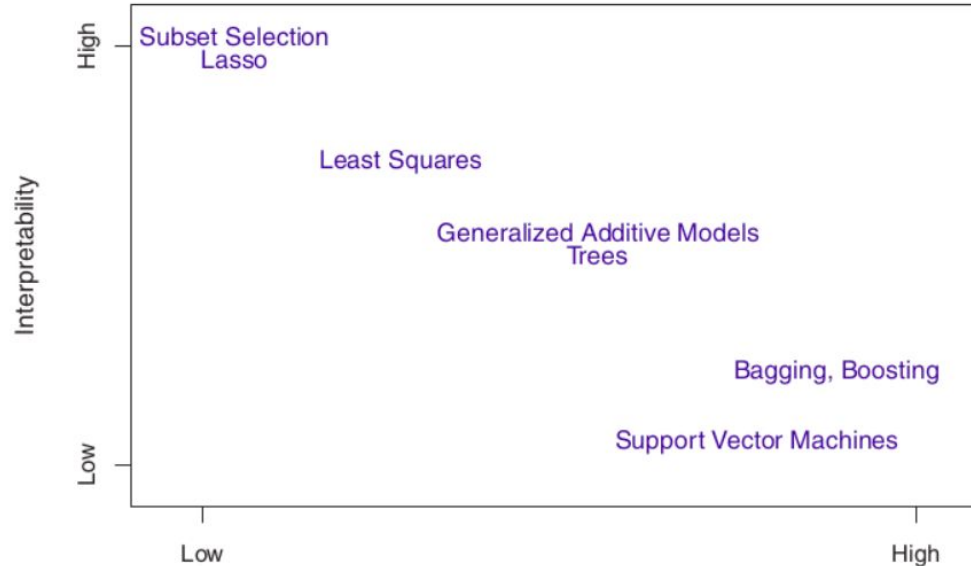
Modelos Flexibles (Complejos): Son modelos que pueden captar relaciones no lineales y patrones muy complejos en grandes volúmenes de datos (como redes neuronales o *random forests*). Suelen ser más precisos en sus predicciones pero actúan como una "**caja negra**", donde no siempre está claro cómo se llega al resultado.

Cuándo priorizar Interpretabilidad: En áreas como medicina o finanzas, donde es vital conocer el mecanismo subyacente o realizar inferencias causales para la toma de decisiones.

Cuándo priorizar Flexibilidad: En tareas de percepción (visión, lenguaje) o fenómenos no estudiados donde el objetivo principal es la exactitud de la predicción por sobre la explicación del proceso.

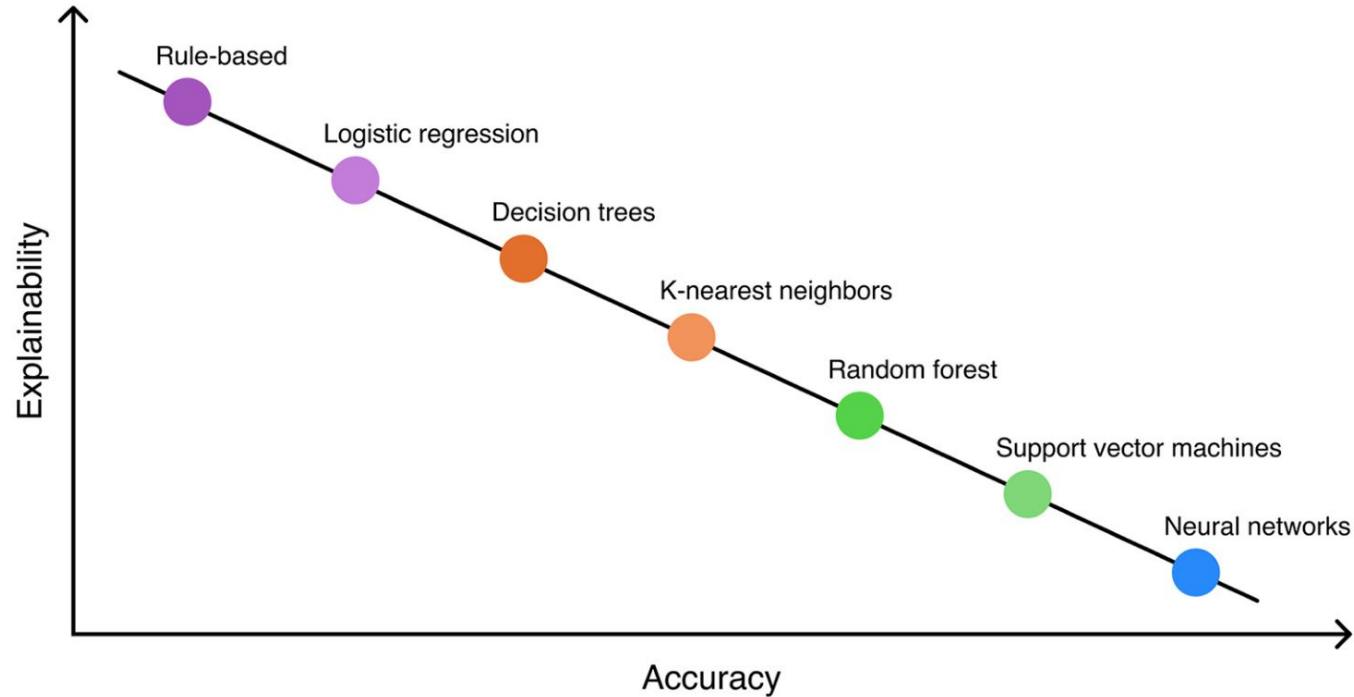
Interpretabilidad vs Flexibilidad

Modelos que son más simples pero pueden ser más imprecisos



Modelos que son poco interpretables pero que pueden ser más precisos

Interpretabilidad de modelos



Parámetros vs hiperparámetros

Parámetros:

- **Definición:** Son valores internos que el modelo aprende a partir de los datos durante el proceso de entrenamiento.
- **Ajuste:** Se ajustan automáticamente a medida que el modelo entrena, optimizándose para minimizar el error o maximizar la precisión.
- **Ejemplo:** En una regresión lineal, los coeficientes (pendiente y la intersección) son parámetros que se aprenden durante el entrenamiento.

Hiperparámetros:

- **Definición:** Son configuraciones que se establecen antes del proceso de entrenamiento y determinan el comportamiento del modelo. No se aprenden a partir de los datos, sino que los define el usuario.
- **Ajuste:** Se ajustan manualmente o mediante técnicas de optimización, como la búsqueda de cuadrícula (grid search) o búsqueda aleatoria (random search).
- **Ejemplo:** En un árbol de decisión, la profundidad máxima del árbol o el número mínimo de muestras para dividir un nodo son hiperparámetros.

Parámetros: Se **aprenden** durante el entrenamiento del modelo.

Hiperparámetros: Se **definen** antes del entrenamiento y controlan cómo se comporta el modelo durante el aprendizaje.

Software

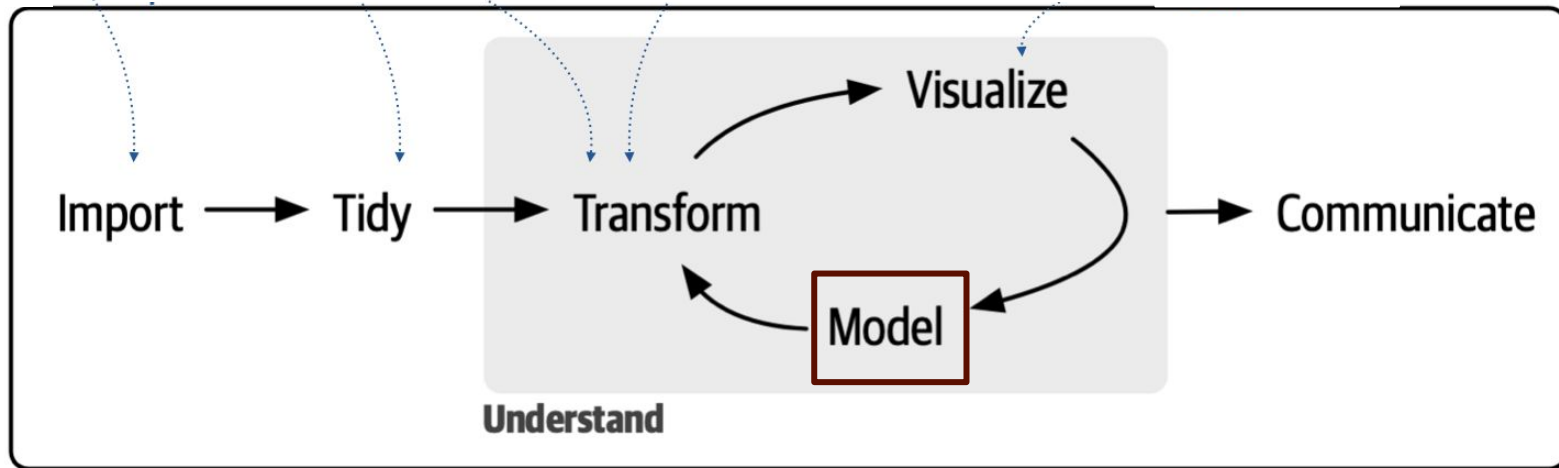
Flujo de trabajo en ciencia de datos



matplotlib

seaborn

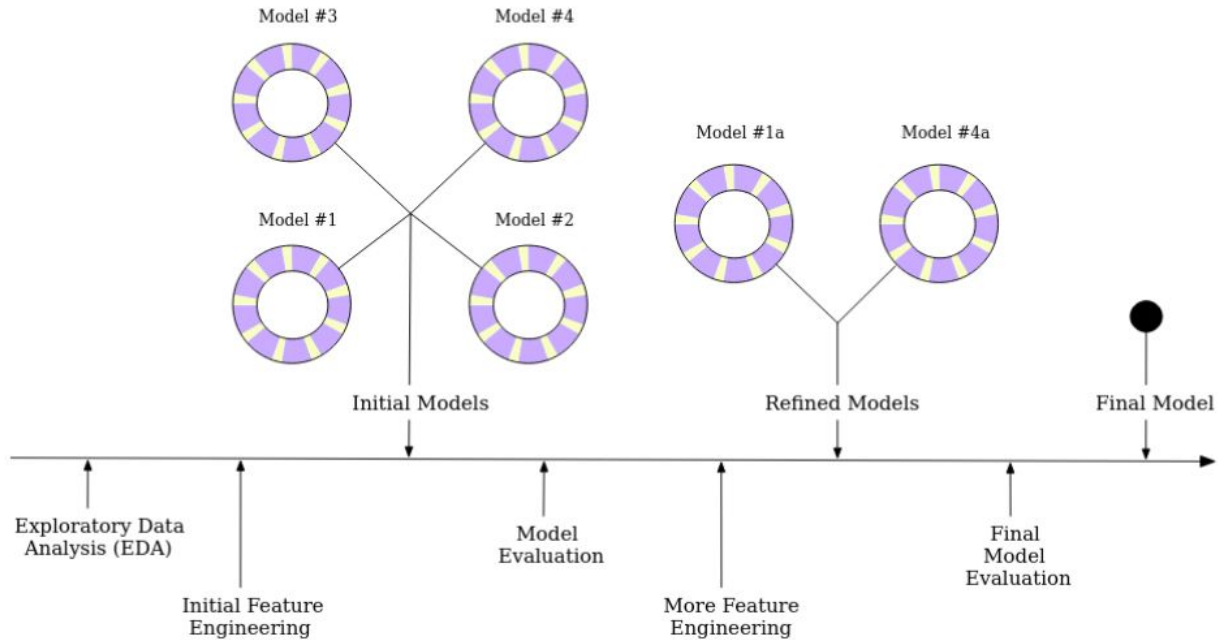
pandas pandas pandas



Program



Etapas del modelado de datos



Aprendizaje automático a nivel empresarial

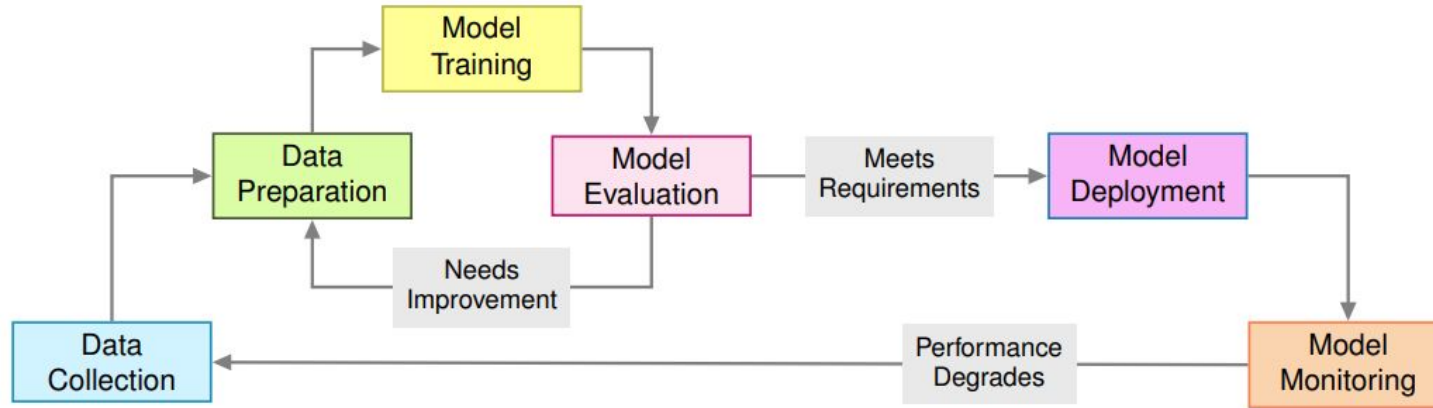
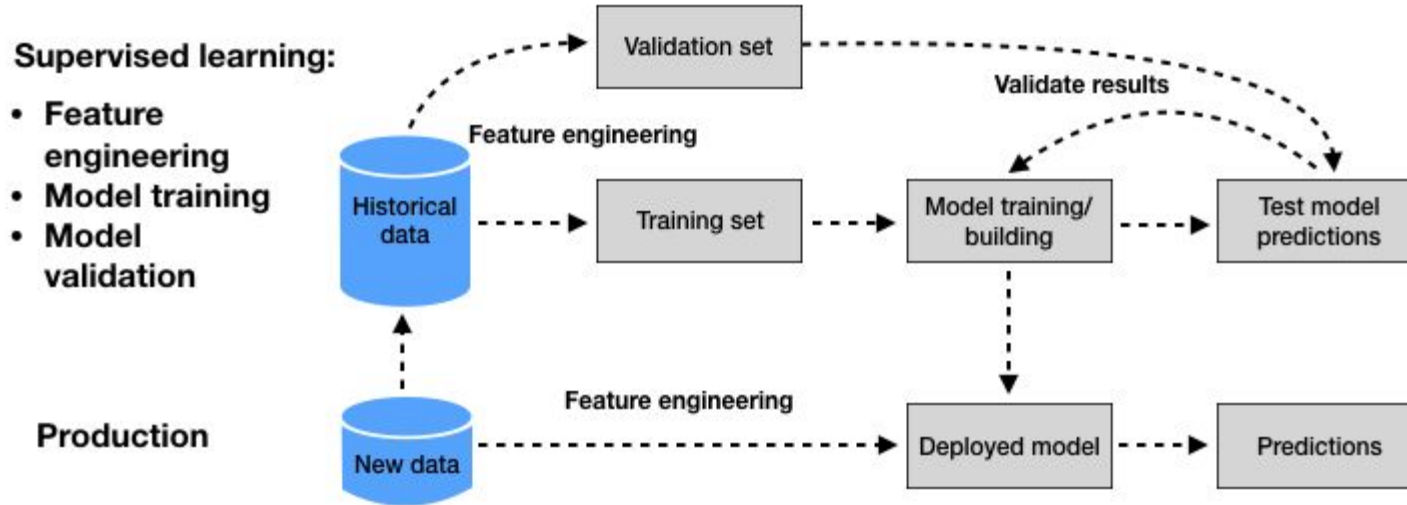


Figure 1.4: ML System Lifecycle: Continuous iteration defines successful machine learning systems, requiring feedback loops to refine models and address performance degradation across data collection, model training, evaluation, and deployment. This cyclical process contrasts with traditional software development and emphasizes the importance of ongoing monitoring and adaptation to maintain system reliability and accuracy in dynamic environments.

Aprendizaje automático a nivel empresarial




[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)



scikit-learn

Machine Learning in Python

[Getting Started](#)
[Release Highlights for 1.8](#)

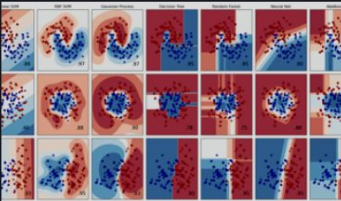
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [logistic regression](#), and [more...](#)



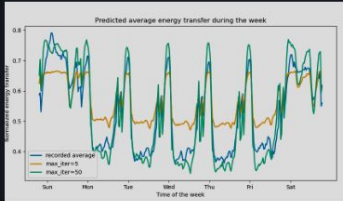
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, stock prices.

Algorithms: [Gradient boosting](#), [nearest neighbors](#), [random forest](#), [ridge](#), and [more...](#)



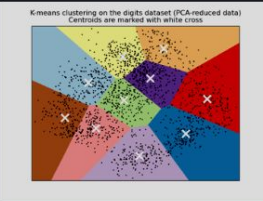
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, grouping experiment outcomes.

Algorithms: [k-Means](#), [HDBSCAN](#), [hierarchical clustering](#), and [more...](#)



Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, increased efficiency.

Algorithms: [PCA](#), [feature selection](#), [non-negative matrix factorization](#), and [more...](#)

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning.

Algorithms: [Grid search](#), [cross validation](#), [metrics](#), and [more...](#)

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: [Preprocessing](#), [feature extraction](#), and [more...](#)



Otras librerías importantes



NumPy

matplotlib



pandas



seaborn

PyTorch



Keras



Realizar un EDA (análisis exploratorio) de los datasets:

1. Stroke dataset
2. Work from home burnout dataset

Utilizar Python y las librerías: numpy y pandas (para manipular datos y explorar el dataset), matplotlib y seaborn (para realizar gráficos).

Objetivos:

1. Identificar la variable target (objetivo)
2. Ver si hay valores faltantes, ver las distribuciones de las variables, ver comportamiento bivariado, correlaciones, posible relación lineal entre variables.

- Luis Serrano. Grokking Machine learning. Capítulo 1
- Andrew Trask. Grokking deep learning. Capítulo 2
- <http://www2.math.uu.se/~thulin/mm/breiman.pdf>
- Machine Learning Engineering
- Introduction to statistical learning
- Machine Learning Bookcamp
- <https://towardsdatascience.com/why-most-cross-validation-visualizations-are-wrong-and-how-to-fix-them-bdabbba74e263/>
- <https://towardsdatascience.com/model-validation-techniques-explained-a-visual-guide-with-code-examples-eb13bbdc8f88/>